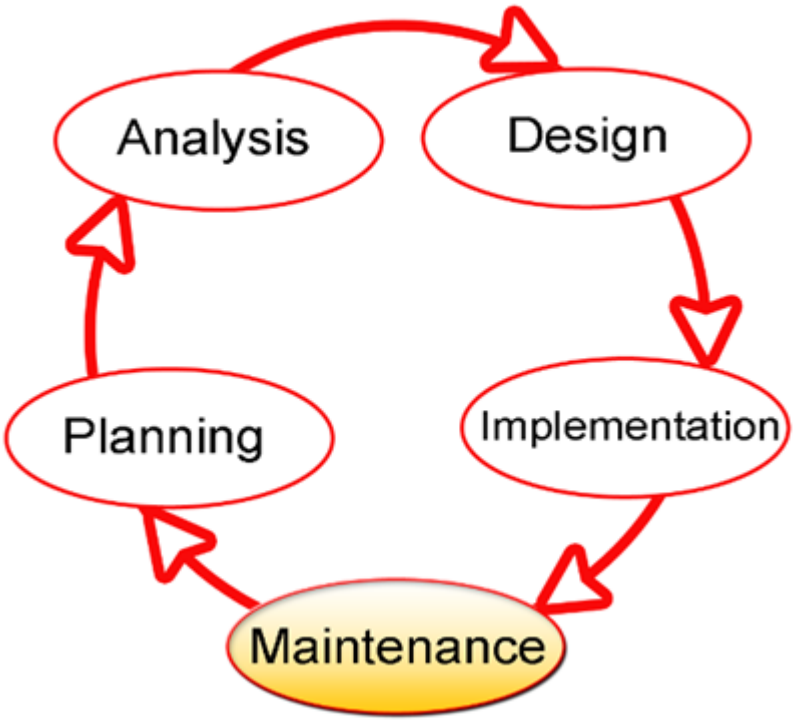
Accessibility automation: Integrate accessibility testing in the development lifecycle.

Development Lifecycle

Developing software is a big task. There can be a lot of wasted time if you do not know what you are doing.

When building software, developers often refer to the Software Development Life Cycle (SDLC).



*Src: By Dzonatas - Own work, CC BY-SA 3.0, **https://commons.wikimedia.org/w/index.php?curid=4376189 (https://commons.wikimedia.org/w/index.php?curid=4376189)***

The SDLC helps developers stay on track during production. Depending on your preferred model, you will choose one of the following SDLC models for your project:

Waterfall

The waterfall development method can only continue to the next phase once the previous phase is finished.

You would add your accessibility testing at the testing phase.

Learn more about waterfall methodology here (https://www.youtube.com/watch?v=bNLcRdrSQAU)

Agile

The agile development method takes a project and breaks it down into small individual parts that is assigned to teams. Each team works on their project to get a chunk of a product out in a short turn around time. It then can be improved upon and get feedback on its development from stakeholders.

Automated accessibility testing can be done after each sprint to work through some accessibility issues.

Before every release a manual test should be done prior to releasing the software.

Learn more about agile development here (https://www.youtube.com/watch?v=1iccpf2eN1Q)

Spiral

The spiral development method has 4 phases that it cycles through until the project is completed.

All manual and automated tests would be done at the end of phase 3 which is the testing section.

Learn more about spiral development here (https://www.youtube.com/watch?v=mp22SDTnsQQ)

V-model

The V-model development method has 3 sets of processes:

1. verification process which sets up the design and plan for the software
2. The coding process which produces the code
3. Validation process which tests several different compliances.

Accessibility testing would be done in the testing process of the v-model cycle.

For more information and comparison to agile click here (https://www.youtube.com/watch?v=wYeugavTPS0)

Most software life cycles will have automated linters, bug checkers and testing tools that get applied during your code before it gets integrated with the rest of the team's code. Having automated accessibility test here would be a good idea to reduce the number of errors that will build up.

As for manual testing, this should always be done before releases as they take more time to complete.

© Algonquin College 2022

Automated testing tools demonstration

In the previous module you learned the importance of having an accessible website and how to use the WCAG (Web Content Accessibility Guidelines) to improve your site.
In this section you will learn how to perform automated tests on your website and how to read the report.

As a note before you begin, automated tools are just one part of your testing strategy. Manual testing should always follow automated testing.

There are things that automated tools at this point are missing, like common sense.
For example, if you have an image on your website you need an ALT text attribute to give a meaningful message to screen readers so people with impaired vision can interact with that image.

An automated tester will only see if an ALT message is present, not if it is meaningful.

Automated testing has been around since 1996 and has been improving quality assurance and testing. Automation has a lot of benefits to it, some include:

- Easy to reproduce the same test at every stage of development along a production lifecycle
- Once setup or made, they are easy to run and produce quick results
- You do not need a deep level of accessibility knowledge to run the test and view the results

There are downsides to automated testing as well such as:

- Automated tests only cover 30% of WCAG success criteria
- They report false positives (Like the ALT example above)
- You may need different testing software for different success criteria

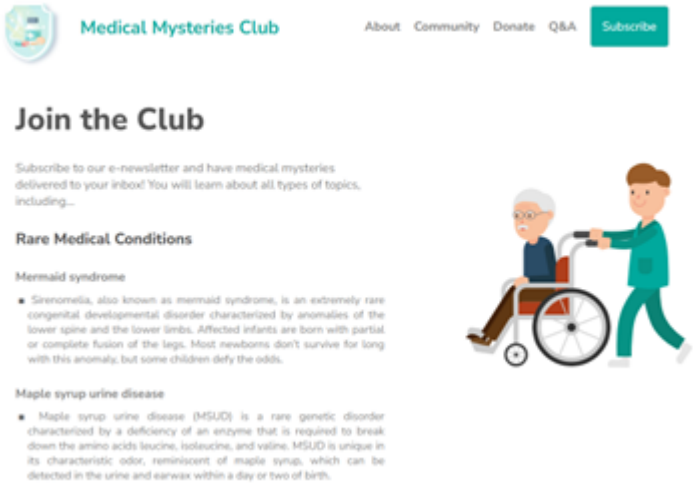
There are **plenty of automated (https://www.w3.org/WAI/ER/tools/)** tools to choose from and each one will depend on what you want to test. For example, did you want to test contrast on your page, did you want to test on a specific browser, etc.

For more information on how to select the correct tool check out W3C's writeup on evaluation tools: **https://www.w3.org/WAI/test-evaluate/tools/selecting/ (https://www.w3.org/WAI/test-evaluate/tools/selecting/)**

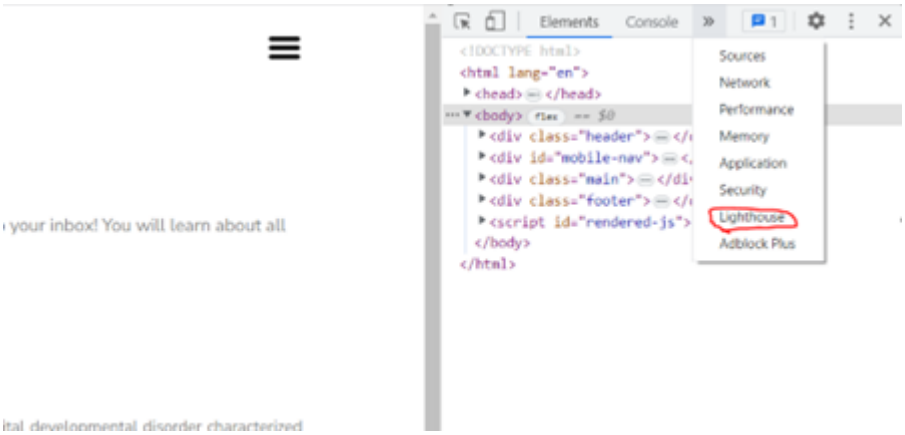
For this section's example you will be using **Google Chrome's Lighthouse (https://chrome.google.com/webstore/detail/lighthouse/blipmdconlkpinefehnmjammfjpmpbjk)** extension.

Once installed you will follow the following steps:

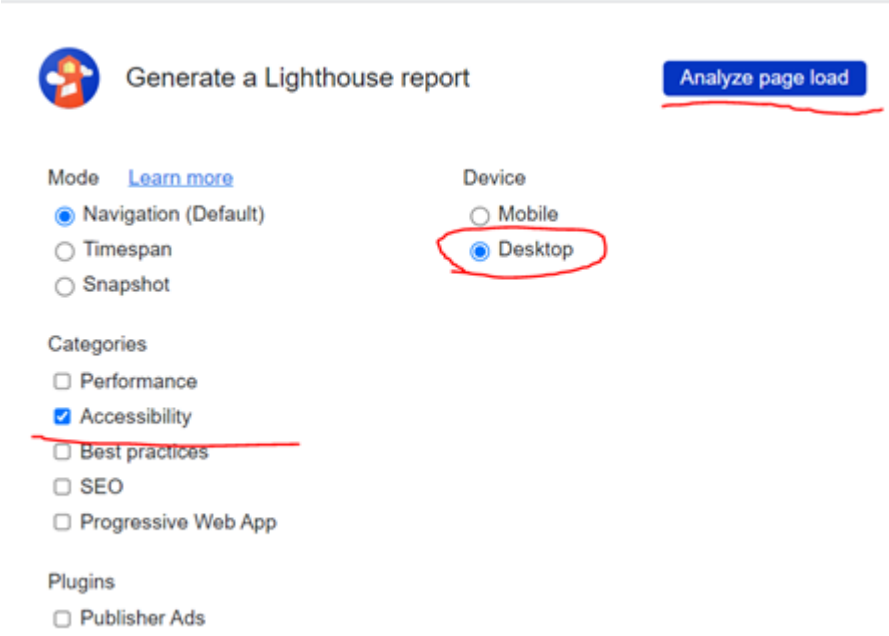
Navigate to this CodePen example of a website that needs improvement. This shows all the code that the webpage has. Also open the Debug version of it to remove the code, as this is what needs testing.



Open Up your dev tools by pressing F12 on your keyboard and navigate to Accessibility.



Set the Categories to Accessibility, Device to Desktop then Click the Analyze page load



Lighthouse will take a minute to process the website. Once finished it will give a score to the website and information about the changes to the site that lighthouse has detected.

Now that you have the error list, you should fix them to improve the accessibility. For a more explained version of the fixes needed see the following website **https://web.dev/learn/accessibility/test-automated/#step-5 (https://web.dev/learn/accessibility/test-automated/#step-5)**

Here is a link to the site with all the suggested fixes changed. Try running lighthouse to check the accessibility automated test results: **https://cdpn.io/pen/debug/PoBZgrW (https://cdpn.io/pen/debug/PoBZgrW)**

As you can see, automated tools can help improve a portion of a website's accessibility issues. In the next section you will cover manual testing.

© Algonquin College 2022

Manual testing strategies built upon automated testing

As discussed in the previous section, automated testing is only around 1/3rd of all the testing you need on your website. Technology is improving and there may be a higher percentage of what automated test can do in the future, but for now manual testing is still the best way to test a site.

Manual tests benefit from:

- Being quick and straightforward to do certain tests
- Have a higher chance of finding more issues than automated tests alone
- Very little experience and tools are needed to complete them

Unfortunately manual tests suffer from:

- Difficulty in scaling (if a website gets bigger the testing time goes up faster)
- Can take more time and complex than automated tests
- Certain tests need more expertise to run them or interpret results.

What can automated tools detect vs manual testing you may be thinking. The following list breaks down the difference.

Can be automated	Can't be automated
Color contrast of text on solid backgrounds	Color contrast of text on gradients/images
Image alternative text exists	Image alternative text is accurate and is properly assigned
Headings, lists, and landmarks exist	Headings, lists, and landmarks are correctly marked-up and all elements are accounted for
ARIA is present	ARIA is being used appropriately and applied to the correct element(s)
Identifying keyboard-focusable elements	Which elements are missing keyboard focus, the focus order makes logical sense, and the focus indicator is visible

iFrame title detection	iFrame, the focus order makes logical sense, and the focus indicator is visible
Video element is present	Video element has appropriate alternative media present (such as captions and transcripts)

Src: **https://web.dev/learn/accessibility/test-manual/ (https://web.dev/learn/accessibility/test-manual/)**

When performing manual test you need to focus on what you want to test. There are many tests that can be performed on various features of your site.

In the following tests you will be testing the following areas: keyboard checks, content checks, and visually focused reviews. Tests are not limited to theses, but they are a great start.

Keyboard Checks

According to intopia, in 2022 61% of critical failures in audits were caused by elements on a webpage that were not accessible via a keyboard.

This affects users who are: sighted keyboard-only users, low-vision/blind screen reader users, and people who use voice recognition software.

The focus of keyboard tests can answer:

- Are there any mouse only features on your web page?
- Does tabbing follow a natural flow of the webpage?
- Can you always see where your keyboards focus is?
- Are there any elements that should not have a focus trap?
- Are you able to navigate behind/around a focus trap?
- After closing a focused element, does focus return to a logical spot?

For testing the keyboard you need to know how to navigate and interact with a site using the keyboard.

The following table has some common key controls.

Key	Interaction
Tab	Moves focus to next interactable
Shift+tab	Moves focus backwards

Return/enter key	Activate the element that is focused (like clicking on a link)
Arrow keys	Cycles through related controls (like dropdown menus or radio buttons)
Spacebar	Moves page down or toggles states (like checkboxes)
Shift + Spacebar	Moves the page up
Escape	Closes interactive elements (like focus windows)

Visual Checks

When doing visual checks you will be focusing on the visual elements on a page and use tools like a magnifier or zoom to check the website.

A visual check test shows you:

- Are there any elements that look like a certain element, but are not (p tags that act as headings, or other structures that aren't what they appear)?
- Are there any elements that do not meet the seizure guidelines?
- Do any images or gradients have contrast issues that an automated test not catch?
- Is there a consistency to form inputs and links on the site?
- Do magnifiers or browser zoom show all content?
- Is there good spacing or letters, words, lines, and paragraphs?

Content Checks

A content check looks at what they webpage's words are. You need to check if it makes sense for other users.

Content checks show you:

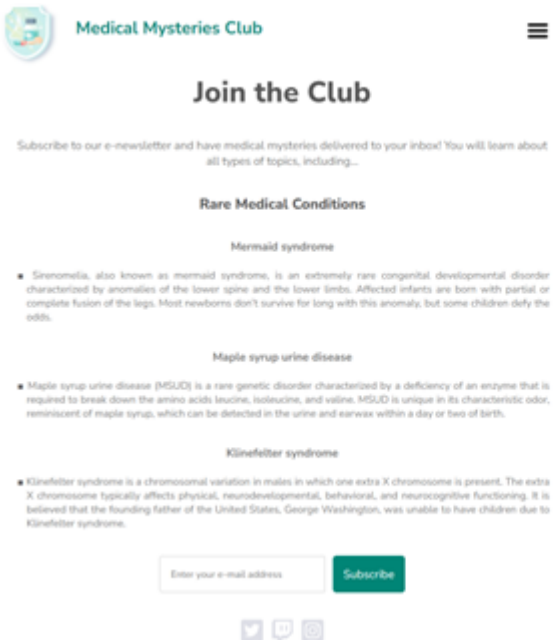
- Are acronyms spelled out first before they are used and is the site in plain language?
- Do the image alternatives describe what is being shown or are useful?
- Is colour used to describe a difference in meaning or information? Can something else be used

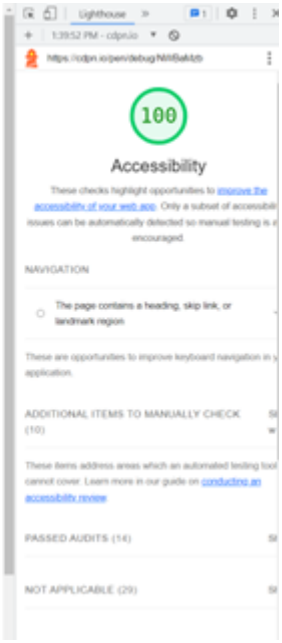
instead? (Ex. Pros and Cons being only highlighted in reds and greens.)

- Do links describe what they are, or does it have a generic link like “Click this” or “Read More”?
- Are there language changes in the page? (ex. French to English)
- Do the page titles, headings and form labels describe what is being shown?

Manual Test Demo:

The following link takes you to the **CodePen Debug mode (https://cdpn.io/pen/debug/NWBaMzb)** that you used before. As you can see in Fig. 4.4, it passes the lighthouse automated tests, but there are still a bunch of accessibility issues.





To Start:

You will start doing manual test to see what is wrong.
Begin by using your keyboard controls. Use the key commands mentioned above to navigate the site.

Are there any issues? Where are you currently focused on?

How are the links? Are you able to visually see any difference between them and the site?

For a full breakdown of the manual test follow along with the **following website. (https://web.dev/learn/accessibility/test-manual/#demo-manual-test)**

When you are finished your manual test, **the website should look like this. (https://codepen.io/web-dot-dev/pen/eYjZdve)**

© Algonquin College 2022

Strategies

In this module you will learn all about accessibility testing. You will learn how manual and automated tests are performed and how certain bugs can be caught while testing.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Use manual testing to remove visual, content and keyboard issues

02 Use automated test to reduce testing workloads

03 Apply automated, manual and screen reader tests to the software development lifecycle

Key Terms and Concepts

1. Automated tests: Tests that can be done by the computer automatically
2. Focus Trap: A HTML element that takes the current focus. Sometimes it prevents/hinders navigation if implemented incorrectly
3. Lighthouse: An addon/extension of chrome that can run multiple automated tests
4. Manual Tests: Tests that must be completed by a human being
5. NVDA: Non-Visual Desktop Access, a piece of screen reading software used in this module
6. SDLC: Software Development Life Cycle: The process of developing a piece of software from start to finish

Screen reader testing

In this section you will be experiencing navigating a website using a screen reader.
In order to try testing a website you need to download and install a piece of software that can read screens. You will need **Non-Visual Desktop Access (NVDA). (https://www.nvaccess.org/)**

This software is a free open-source screen reader that anyone has access to. It runs on the help of developers all over the world and donations from people and companies that want to support their efforts.

NVDA works with Chrome and Firefox browsers on windows and multiple other desktop applications.

To download it, follow this link. (https://www.nvaccess.org/download/) Note: if you only want to use it for this example, choose the “Continue Running” option. This will not download it, but it will run it as a temporary executable.

Once run you should here the narrator speaking along.

The following is the key controls to use the screen reader.

Key Command	Function
insert	**NVDA Command Key**
ctrl	**Stop audio**
↓ or ↑	**Read Next/Prev**
NVDA(Insert) + ↓	**Start Reading**
NVDA+F7	**Elements List**
D	**Landmarks toggle**
K	**Links Control**
F	**Forms Controls**
T	**Tables Controls**
NVDA+ Alt + ↓ ↑ ← →	**Navigate Tables**

Now that you have the controls to navigate a website using just your keyboard and a screen reader you can **test the CodePen website (https://cdpn.io/pen/debug/eYjZdve)** for Screen Reader navigation.

For a follow along guide **use the following website (https://web.dev/learn/accessibility/test-assistive-technology/#step-2)** to pickup all the problems with the site.

As you have seen firsthand, enabling better navigation and clarity on websites elements goes a long way in helping people who use screen readers to interact with those sites.

© Algonquin College 2022

User functional testing

User Acceptance Tests

After the last 3 section, you have performed:

- Automated tests to catch common syntax and structural errors
- Manual tests to catch visual, keyboard, and content errors
- Screen reader tests to improve screen reader access

What you are left with is a website without any errors that can cause problems for users with disabilities, however there are more tests that should be run.

In this section you will learn about functional tests.
Functional tests provide feedback on how a user actually uses the website.

The types of testing are as follows:

Smoke Testing

Sanity Testing

Integration Testing

Regression Testing

UI/UX Testing

User Acceptance tests

Since this is an accessibility course and not specifically a testing course, the focus will be on User Acceptance Tests. For more **information on the other tests, feel free to follow this link. (https://dzone.com/articles/accessibility-testing-vs-functional-testing)**

The primary goal of a User Acceptance Test(UAT) is to determine if your website can be used, both by user with and without disabilities. You have put in so much work on a website to make it accessible, but if no one wants to use it or they don't know how to use it then it is all for nothing.

Since you designed the website, you know best how to use it. You will likely be using it the way you intended it, however others likely won't.

A funny example of this is this shape sorting video: **https://youtube.com/shorts/jkz7bnYfuOI (https://youtube.com/shorts/jkz7bnYfuOI)**

Even though the design looks clear and straight forward, it can be easily misinterpreted.

How to perform a UAT

There are 5 steps to perform a UAT:

- Plan
- Identify and create real-world test scenarios
- Select the testing team
- Perform the tests and document results
- Update code, retest and sign off

Plan

The planning stage is just as the name suggests, you plan. You need to figure out how much time you have for testing (if you have a specific delivery date of the project), what business requirements you have (like money, resources, non-disclosure agreements), and finally what strategies you'll use.

If this is a large company you are doing the testing for, then this plan will have to be written down before performing it.

Identify and create real-world test scenarios

This phase of the testing process wants you to find all the cases you want to test. An example of this is if you have a shopping website, then you may want to test user's logging in, users adding items to the cart, and users purchasing a product.

This is where you get to synthesize your site down to its core functionalities.

Finally you want to have as many cases as possible where a problem may arise in use.

Select the testing team

This will be the most important step in this process. Since you are designing with accessibility in mind, you will want people with different disabilities to test your site. They will want to see if it is usable to them and their needs.

Your best choice is a diverse selection of people with different backgrounds.
Ensure you inform your users on how to detect and report issues (how to reproduce them being a big part of that)

Perform the tests and document results

At this stage of the testing process your testing team performs all the steps outlined in the second stage. The testers will log any bugs or issues found into a bug tracker with steps to reproduce them and who found them. Depending on the size of your list of test cases, you may want to divvy them out between testers or have all testers perform all tests. It really depends on the size of your software, the time you planned and budget.

Update code, retest and sign off

At this stage of the process you address any bugs/issues found in your code and fix them. Once fixed, you must retest the test cases that originally failed to see if the fixes solved the errors, caused new problems or the issues or bugs persist.
Once the bugs have been solved and retested you need the testers who found and tested and reported the bug sign off on the bug being fixed.

Once all five steps are finished you should be free to release a well-made website.

Lastly, ensure you maintain good communication throughout the testing process.

© Algonquin College 2022

Acceptable uses of tabindex attribute

How to properly use tab indexes

In a previous section you learned what the tab index is and how it affects the tabbed flow order. There are common mistakes that happen when using tabindex.

Mistakes

- Manually setting the tab order
- Implementing positive tabindex values
- Focusable non-interactive elements
- Having a non interactive child element with a tabindex attached inside an interactive element.

Uses for tabindex

- Making JavaScript widgets and charts interactive
- Having scrollable overflow containers
- If you really need to make an HTML element focusable

Mistakes

Manually setting the tab order
To sum up this mistake, this makes things very redundant. You will spend extra time manually getting interactive elements in sequence when, by default, the natural flow of a web page goes sequentially already.

Bad:

```
<a href="#Link1" tabindex="1">Link 1</a>
<a href="#Link2" tabindex="2">Link 2</a>
<a href="#Link3" tabindex="3">Link 3</a>
```

Good:

```
<a href="#Link1">Link 1</a>
<a href="#Link2">Link 2</a>
<a href="#Link3">Link 3</a>
```

You end up doing more work when manually adding them. There is also more room for a random jump in your tab focus to happen.

Implementing positive tabindex values

This mistake is really just a flow issue. Having user's tab focus jump around the page is not very helpful.

It is best to let the webpage natural sequence dictate where the flow of the page should go.

Bad:

```
<p>
    Here at <a href="OnlineCollege.html">Algonquin College Online</a>
    we learn about <a href="coding.html">coding</a> and
    <a href="cloud.html" tabindex="3">cloud development</a>
</p>
```

Good:

```
<p>
    Here at <a href="OnlineCollege.html">Algonquin College Online</a>
    we learn about <a href="coding.html">coding</a> and
    <a href="cloud.html">cloud development</a>
</p>
```

Focusable non-interactive elements

Making non-interactive elements have focus is not necessarily the best practice. Most non-mouse users rely on tab index to cycle through the clickable/fillable interactive elements. If you have focus on an element that was not interactive, it can be confusing what you are supposed to do with it.

Imagine having a cup that you were not able to use for drinking. It adds ambiguity to your website.

Bad:

```
<p tabindex="0">
    Here at Algonquin College Online
    we learn about coding and cloud
    development
</p>
```

Good:

```
<p>
    Here at Algonquin College Online
    we learn about coding and cloud
    development
</p>
```

There are times when you add a tab index to a div or span, but those usually are programmed to be interactive.

Having a non interactive child element with a tabindex attached inside an interactive element.

This common mistake doesn't harm the webpage but it leads to an odd behaviour.
What is happening is you have an element like a button, which naturally has focus. If you nest a non interactive element inside and apply focus to it using tabindex, you receive double focus on the same element essentially. When the user tries to Tab away from the button, the next element is its child, so the focus still is on the button, but now you cannot interact with it.

Bad:

```
<button type="button">
    <span tabindex="0">Click to go to grades</span>
</button>
```

Good:

```
<button type="button">
    <span>Click to go to grades</span>
</button>
```

Proper uses for tabindex

Making JavaScript widgets and charts interactive

You can apply a negative tabindex to some interactive widgets and components to programmatically set the focus using the **JavaScript focus() method. (https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/focus)**

Visit the link for uses of focus.

Having scrollable overflow containers

An **overflow container (https://www.w3schools.com/css/css_overflow.asp)** is a text area that has been shortened in its physical space on the webpage.

It is a CSS property to limit how much space it consumes on a webpage.

Since this is a non-interactive element that typically doesn't have focus, it actually helps to give it focus as keyboard users who do not use a screen reader need access to that field so they can use the keyboard to scroll through the text.

If you really need to make an HTML element focusable

If there is an element on your page that is very important, and you feel needs focus, then it is your decision to make it focusable.

Just use caution and ensure that you want that element to have focus.

© Algonquin College 2022

Focus order and focus indicator.

Focus Order

You have already learned what focus is when it comes to webpages, and you know how to toggle through focused elements on a webpage. A browser will typically try to read focused elements in sequence, based on where they are in the html code (from top to bottom).

However you may want to manually move focus in a certain way that differs from the regular flow of the HTML code.

Typically you should try to avoid moving focus away from the logical flow unless necessary.

In the first section you learned that the tabindex property allows non-focusable elements to come into focus.

Typically a value of 0 (tabindex= "0") is used to grant focus.

Then to change the order of how focus flows it goes up based on a positive value.
When the tab index's value is 1 (tabindex="1") then that means the element with that property has the HIGHEST priority.

Then the next number higher than 1 will have the next highest priority.
Example. A tab index of 5 has a lower priority than an element with an index of 3.

Finally if you want an element to be focusable, but not accessible by keyboard, then you can set the tabindex to -1 (tabindex="-1"). This typically is not used, unless you want to programmatically grant it focus later using JavaScript.

The following example site (https://cdpn.io/pen/debug/QWxWwNM) shows how focus order by tab index is achieved.

If you want to see how the HTML code looks, you can press F12 to open up the developer tools or you can right click and select "View Page Source".

For a review of tab focus order, watch the following video: **https://www.youtube.com/watch?v=Pe0Ce1WtnUM (https://www.youtube.com/watch?v=Pe0Ce1WtnUM)**

Skip Links

Now focus is a handy thing to have that let's user with disabilities navigate a website. However there may be times when that user would like to jump straight to the content rather than navigating through all the links to other webpages.

This is where skip links come in. A skip link is typically the first link on a website. Its whole purpose is to skip focus from the navigation links to the content.

They will typically use a sections ID as part of an anchor to jump to that spot.

Link to content

This link is typically made just for keyboard users so you can have it visible or only visible when tabbed to.

Visit the following webpage example (https://cdpn.io/pen/debug/LYrYEGo) and try to tab through the elements.

What you should see when you first tab through is a link that appears at the top that is labelled "skip to content"
It will disappear after you tab past it. This is done through CSS properties.

One thing to keep in mind when building a skip link, make sure you apply a label or write a meaningful message in between the anchor tags so screen reader users can understand the link's purpose.

Focus Indicator

One thing you may have noticed on some of the websites you have been tabbing through, is how the focus looks. What do you actually see when you are tabbing through links and elements?

Setting up a good visual focus indicator goes a long way for users with low vision. The WCAG standard for focus indicator contrast is to have a contrast ratio of at least 3:1. This is between your background colour and the focus indicator.

To test this out, go to the following website and try changing the foreground and background colours.

You will be able to see if the two colours you selected meet WCAGs contrast ratio for both AA and AAA compliance.

To review what a focus indicator is watch the following video: **https://www.youtube.com/watch?v=Pe0Ce1WtnUM (https://www.youtube.com/watch?v=Pe0Ce1WtnUM)**

Now that you have your compliant colours, how do you apply them to your focus indicator?

To alter a focus indicator you can apply it in your CSS page using the "focus" pseudo-class.
e.g.

You can remove an outline from a link, BUT it is highly recommended not to and goes against WCAG standards.
If you want to though, you set the outline to "none"

Finally if you want to re-apply your browser's default style you would add the following:

Src: **https://web.dev/learn/accessibility/focus/ (https://web.dev/learn/accessibility/focus/)**

You can see some different focus styles applied here.

© Algonquin College 2022

Keyboard operability for web pages and widgets

In the first module you covered what accessibility is and how it affects users all over the world. Many users who face disabilities on websites cannot use a mouse. This can include, but not limited to users who:

- Are blind
- Low vision
- Motor function impairment
- Prefer keyboard use

In the previous module you did some keyboard testing examples and learned a little of how navigation in that is made.

Keyboard navigation is used mostly by tabbing focus through a web document.
Tabbing involves tapping the “Tab” key to move a virtual cursor through different elements on a webpage. As focus changes a screen reader will start from wherever that focus is currently. A user can then use “space bar” to select, unselect, or interact with that element.

If a developer has coded a website with accessibility in mind, then a “Focus indicator” should be seen as you tab through elements.



What is focus?

As mentioned above, tabbing changes the focus of the webpage, but what is focus?

Focus is the current element that is selected. You use it every time you log into a website. When a webpage asks you for your login credentials, how do you know what field you are entering your information into? Focus solves this issue by highlighting and prompting the cursor to the correct element like the username field.

Not every element on a webpage will have focus set on it by default.

Elements that have a focus state by default are the following tags:

- <a>
- <button>
- <input>
- <textareas>

However, focus can be applied to any tag by adding an attribute to the tag.
Both tabindex and contenteditable attributes will allow you to tab and gain focus on that element.

The following code sets a div tag to be focusable.

A div tag normally doesn't have focus, but because I have a tab index I now do!

Keyboard Traps

One concern while developing an accessible website is to be aware of keyboard traps.
A keyboard trap is when a user cannot tab through the whole website. The keyboard's focus gets stuck either on an element or set of elements and cannot let the user proceed any further.

However there are other times when you do want to trap focus.

The reason for this would be if you want to trap focus in a modal pop-up window. If focus is not trapped, then the focus can be selecting elements behind the modal window.

The video below has visual examples of how keyboard traps happen, and it also shows code on how to avoid them. Feel free to watch until the 2:14 mark.

- **https://www.youtube.com/watch?v=BoAsayPVogE (https://www.youtube.com/watch?v=BoAsayPVogE)**

© Algonquin College 2022

Management

In the previous module you learned all about manual testing. One aspect of that manual testing was keyboard accessibility. In this module you will learn how accessible code is used to help keyboard users navigate webpages. You will learn how a webpage is setup for keyboard flow and how to change focus of elements while preventing focus traps.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Identify if a webpage has a keyboard trap

02 Use programming properties to control flow of a webpage

03 Alter the default focus styles to be WCAG compliant

Key Terms and Concepts

1. Focus: a visual representation of what a keyboard user current interactive element is
2. Pointer Events: Triggers that are sent to a browser from mouse controls
3. Skip links: A html link usually, the first to be focused, that skips down to the content of a webpage for convenience
4. tabindex: An HTML property that adjusts focus and tab order based of its value

Pointer events

What are Pointer Events?

If you are a mouse user when browsing the internet you will typically interact with buttons and links by clicking them. When you click on those interactive elements you are creating an “event”.

Events can be captured by a browser and then interpreted or coded to do something.

Events can also be suppressed, for example if you have a link placeholder that doesn’t have a finalized link yet, and you don’t want users navigating away from them. You can disable them by using the pointer-events CSS property.

There are currently 2 broadly supported states the pointer event can be: auto or none.
“auto” lets the main functionality or programmed functionality of that element to happen as normal. “none” disables all functionality of those elements from mouse interactions.

For a quick breakdown of how to apply this CSS property to an element watch the following video

- **https://www.youtube.com/watch?v=eh_nsXI66CM (https://www.youtube.com/watch?v=eh_nsXI66CM)**

To summarize, the pointer-events CSS property enables or disables mouse activity from happening on a webpage.

Downsides of Pointer Events

Navigate to the following website **https://cdpn.io/pen/debug/oNbErrK (https://cdpn.io/pen/debug/oNbErrK)**

There will be a button there.

- Try clicking it. What happens?
- Use your keyboard to “tab” navigate to the button then press spacebar. What happens?

In the example, you will see a button on the web page that is disabled to clicking with a mouse.

However, if you navigated with a keyboard, then you will still be able to interact with the button and click it using spacebar.

To a visual user the button seems disabled, however to a keyboard and screen reader user, it is an ordinary button.

Rather than using pointer events to disable a button, it is better to add the “disabled” property directly to that html element.

<!—INSTEAD, DO THIS -->

<button disabled>The button is NOW disabled<button>

- This both disables the keyboard and mouse from interacting with this button.
- A screen reader will announce the button as a “disabled button”
- This is more semantically correct

Ensure you are using pointer-events with accessibility in mind.

© Algonquin College 2022

Content structure; sections, paragraphs, lists, quotes

In the previous section, you learned about the page structure of a webpage and how the regions divide the webpage itself. Along with page structure, there is content structure.
Content structure refers to the actual layout and conveyance of the information you wish to deliver.

There are many ways you can improve the delivery of information to users.

Sections: (https://www.w3schools.com/tags/tag_section.asp)

As discussed in the previous section, HTML sections help break up content into groups of like themed elements. For example, a section that talks about how p tags are used.

Paragraphs: (https://www.w3schools.com/html/html_paragraphs.asp)

The paragraph tag ( ) is used to mark up text and apply a paragraph styling to it. P tags are block tags, which means, if you have a p tag next to a p tag, they will show up on separate lines.

The plan with p tags is to break up text into digestible chunks that is easy to read.

Ex. This is a p tag example on how to use it. If you want another paragraph block, you will use another p tag after you close this one.

Lists:

There are three types of lists in HTML:

- Unordered lists
- Ordered lists
- Description lists

Unordered Lists: (https://www.w3schools.com/tags/tag_ul.asp)

An unordered list is like the list above. It usually uses bullet points but can be changed based on your html code.

They are typically used where the order does not matter. Like a shopping list.
You declare a list by using the <ul> tag (unordered list). Followed by your <li> tags for each item (list item).

Ordered Lists: (https://www.w3schools.com/tags/tag_ol.asp)

Ordered lists contain list items in a specific order. This is usually used for instructions where the order and steps are sequential.

1. Peel and chop Carrots

2. Add carrots to boiling broth

3. Add salt and pepper

For this list you will use an <ol> (ordered list) tag followed by <li> tags.

Description Lists:
A description list usually has a description term with a list of description details.

An example would be:

Accessible by Design
Is a course that is taught at Algonquin College.

The description term is “Accessible by Design” where the description details are “Is a course that is taught at Algonquin College.”

The description lists are made using <dl> tags, with nested <dt> tags for description terms, and <dd> tags for description details.

Lists notes:

You can nest lists inside each other for example an unordered list inside of an ordered list item.

For more details on how lists are used to meet **WCAG standards see the WAI tutorials on lists. (https://www.w3.org/WAI/tutorials/page-structure/content/)**

Quotes:

Quotes are needed when referencing material from an outside source. There are two types of quotes you can use; inline and blockquotes.

Blockquotes:

Blockquotes use the <blockquote> tag to surround a tag since block quotes are typically used for longer chunks of text that are referenced.
Ensure you use the <cite> tag to refer to the referenced source.

Ex. From <cite>Hamlet by William Shakespeare</cite>

<blockquote>

“To be, or not to be: that is the question:

Whether ’tis nobler in the mind to suffer

The slings and arrows of outrageous fortune,

Or to take arms against a sea of troubles,

And by opposing end them. To die: to sleep;”

</blockquote>

Inline quotes:

Inline quotes are typically used in the same line they are being referred to.

They use the <q> tag and are used inside a tag.
William Shakespeare's most famous quote is <q>" To be, or not to be: that is the question"</q>

Quote tags are used so AT can read where the start and end of the quote are and convey that to the user.

Good practices:

If you would like a review on how content structure **can improve a user's experience check out this video. (https://www.youtube.com/watch?v=-w80_z3nnxg)**

In the video the developer is using a WordPress content management system for their website, however the practices that they are using for layout and text breakdown can still be applied to HTML as well.

© Algonquin College 2022

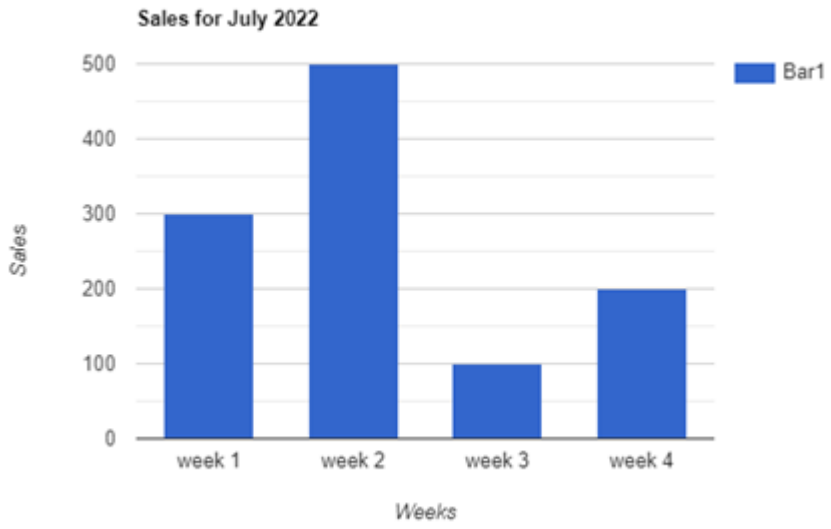
Figures

What are Figures?

Figures in HTML usually show information that the main content can refer to. They will display lists, images, tables, or other content. They are a container that can hold elements that they refer to.

To add a figure you need two tags, a figure and figcaption tag.

The image is referring to the sales_graph.jpg file in the following code.



The figcaption is nested inside of the figure tag.

If you find that you also need to add a longer description for the figure you can alter the img tag's "**longdesc (https://www.w3schools.com/tags/att_img_longdesc.asp)**" attribute.

Ensure you also apply aria labels to the figure tag so AT users can quickly understand what that figure is and if it is relevant to them.

Images

Images are a great way to convey information to a user in a different way than writing. However, it is important that the images are used with an alt text to allow screen reader users the ability to understand what the image is and why it is important.

Video 6.1 describes good alt text practices.

There are many use cases to improve image alt text, for example is the image important to the content topic, or is it purely decorative? If its decorative, then you may not need alt text.

For a full breakdown on creating accessible alt text, view WAI's image tutorials. (https://www.w3.org/WAI/tutorials/images/)

For a great talk about alt text check out the video below

- **https://www.youtube.com/watch?v=flf2vS0IoRs (https://www.youtube.com/watch?v=flf2vS0IoRs)**

© Algonquin College 2022

Links & Buttons

Links and buttons are both interactable elements that users can use to go to another page or activate something in the webpage.

Links: (https://www.w3schools.com/tags/tag_a.asp)

A link is an element that keyboard users can tab to. They can activate a link using the “enter” key.

A link consists of an <a> tag to generate it on the page. It has several attributes typical ones are href, which is where you want to link to and target which refers to how the link should be opened (new page, same page etc.)

Text for the link will be contained between the opening and closing tags.

 Go to google!

The above would appear like: Go to google!

Buttons: (https://www.w3schools.com/tags/tag_button.asp)

A button is also an interactive element that keyboard users can tab to. Unlike links, users can use space or enter to click the button.

A button acts similar to a link however their action is based off of their “onclick” attributes value. The type specifies what type of button it is.

A button can also have images or text as part of it.

<button type=”button”> Click Me! </button>

For users using AT having an aria label would be best to ensure the users understands what the button is used for.

For more information on how to create buttons for accessible design, watch Video 6.4

Larger buttons and links

There are several ways to improve links and buttons for accessible users such as contrast, roles, and labels.

However one of the best improvements you can make is increase the button size.
Increased size of the button allows users who have low vision to easily see the button and text, but it also improves usability for users with low motor control skills or tremors.

- **https://www.youtube.com/watch?v=CzfKB3PuuIY (https://www.youtube.com/watch?v=CzfKB3PuuIY)**
  (https://www.w3.org/WAI/perspective-videos/controls/)

For more information visit the WAI site on large links, buttons, and controls. (https://www.w3.org/WAI/perspective-videos/controls/)

WCAG SC 2.5.5 (https://www.w3.org/WAI/WCAG21/Understanding/target-size.html)

This success criteria states that a button or link must be at least 44 by 44 CSS pixels. This allows for users to press that button easily but also improves performance on mobile devices where the screen and target size is generally smaller.

© Algonquin College 2022

As you learned in the previous modules, the elements on a webpage should have a clear intent and role to all users. If an element acts like another element and only differs visually It can cause problems and confusion for non-visual users.
In this module you will learn all about semantic HTML elements, what their purpose is and how to use them with accessibility in mind.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Create accessible links and buttons that comply with WCAG standards

02 Design a more accessible and navigable webpage by using correct HTML tags

03 Create greater accessibility through useful ALT text for images and figures

Key Terms and Concepts

1. ALT text: Text that is read by a screen reader to describe an HTML image
2. Headings: Various sized HTML tags from h1, to h6 (largest to smallest) that are used to break down content
3. Non-semantic: An element without a distinct role
4. Semantic: An element is an element/tag that clearly defines its content or purpose
5. Landmarks: Key regions on a webpage that are defined by semantic tags

Page Structure; regions and headings

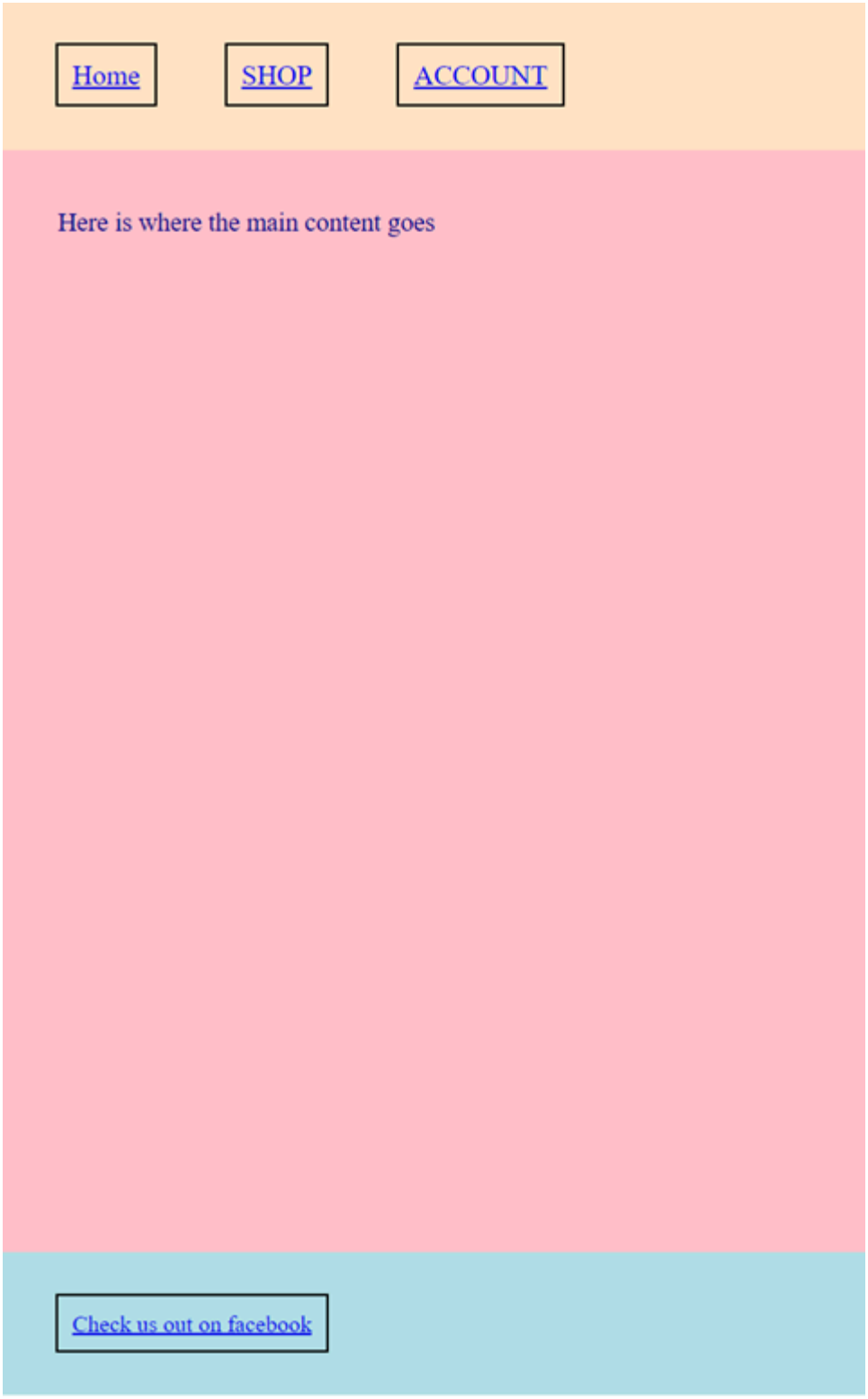
Page Structure

When talking about HTML elements, two types can be observed; Semantic and non-semantic.

- A semantic element is an element/tag that clearly defines its content or purpose. An example would be form, table, or article tags.
- A non-semantic element would be an element without a distinct role.

Example of these elements would be div or span tags.

The idea of semantic page structure is that each element is clearly defined for both visual and non-visual users. If you want to use a non-semantic element you will need to apply extra work to ensure its functionality and roles are clearly defined for non-visual users.



In image above a mock site with limited style has been made.

There are three links on the top navigation bar (yellow section).

The main text area (pink) is where the content would be.

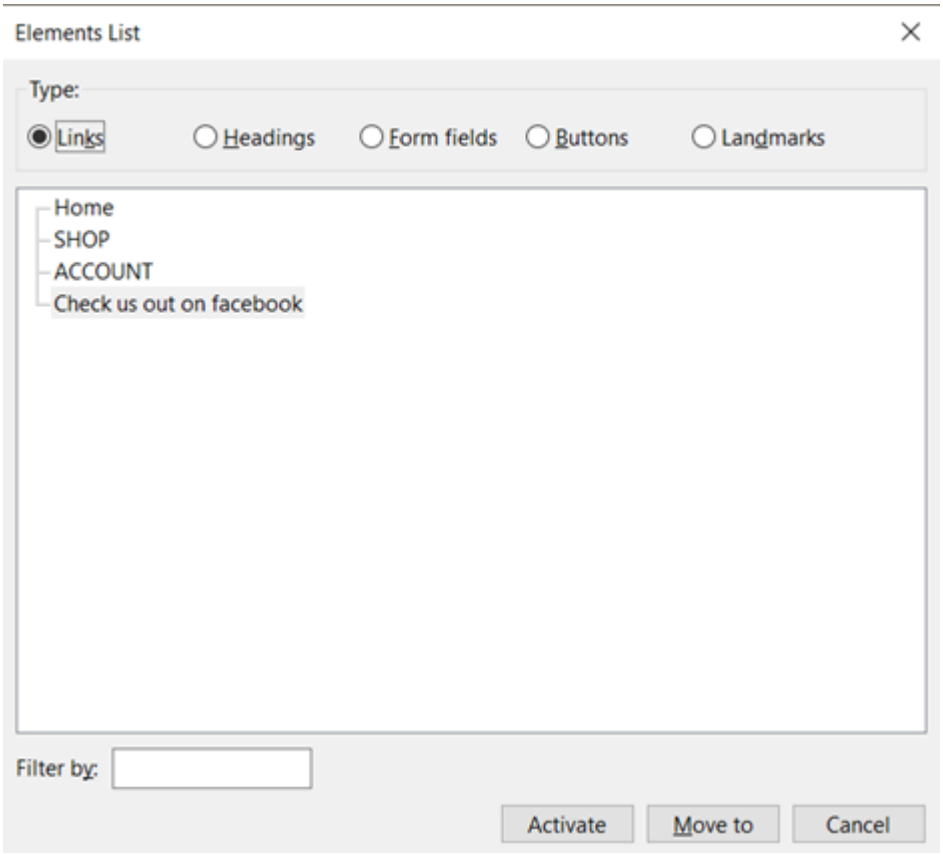
Finally you have a footer along the bottom (blue section)

This would be poor, since the sections are created by the use of div tags and style.

```
<head>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="nav">
    <a href="#1">Home</a>  <a href="#2">SHOP</a>
      <a href="#3">ACCOUNT</a>
</div>
<div class="main">
    Here is where the main content goes</div>
<div class="footer">
    <a href="#4">Check us out on facebook</a></div>
</body>
```

A screen reader would have a challenging time navigating this page since there are no landmark items for quick navigation.

The only quick navigation feature in the list is the links seen in Image 6.3



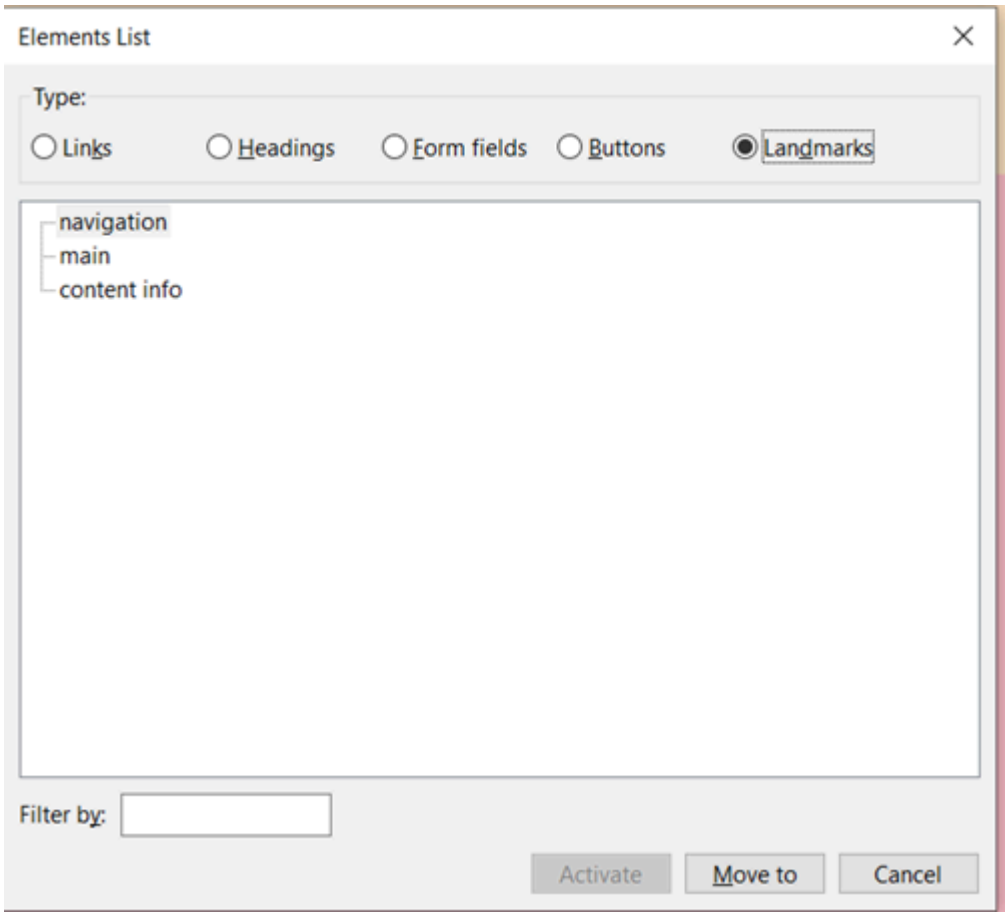
Now that you know there are problems for navigation when using non-semantic HTML elements you can do a change to your webpage to make it mor semantically correct,

Observe the following changes in Image 6.4

```
<head>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <nav class="nav">
        <a href="#1">Home</a>  <a href="#2">SHOP</a>
          <a href="#3">ACCOUNT</a>
    </nav>
    <main class="main">
        Here is where the main content goes</main>
    <footer class="footer">
        <a href="#4">Check us out on facebook</a>
    </footer>
</body>
```

If you run the webpage in a browser using the following changes, visually nothing is different.

However when running your screen reader and you access the elements list (insert+F7) then you will see that the landmarks page now has some quick navigation links. See Image 6.5



In summary, not much work was done to the webpage, and the pages both work the same functionally and visually, however it has improved the website overall for users who use a screen reader.

Regions

The webpage is broken down into different regions. These will show up in the “landmarks” sections of screen reader software.

The following are semantic regions of a webpage with links on how to use them and where they go in a webpage:

- **article (https://www.w3schools.com/tags/tag_article.asp)**
- **aside (https://www.w3schools.com/tags/tag_aside.asp)**
- **details (https://www.w3schools.com/tags/tag_details.asp)**
- **figcaption (https://www.w3schools.com/tags/tag_figcaption.asp)**
- **figure (https://www.w3schools.com/tags/tag_figure.asp)**
- **footer (https://www.w3schools.com/tags/tag_footer.asp)**
- **header (https://www.w3schools.com/tags/tag_header.asp)**
- **main (https://www.w3schools.com/tags/tag_main.asp)**
- **nav (https://www.w3schools.com/tags/tag_nav.asp)**
- **section (https://www.w3schools.com/tags/tag_section.asp)**
- **summary (https://www.w3schools.com/tags/tag_summary.asp)**

Webpages are sometimes broken down like the Image below using the discussed regions above.

The tags not seen here are situational and will usually be nested in some of the main content areas.

Headers:

A header will sometimes act like a banner at the top of the webpage. It usually has a website's logo, search bar and other important buttons.

Nav:

A navigation section usually contains link to other sections of the webpage or website. Labels are suggested if you have several navigation menus to allow them to be differentiated.

Section:

A section is a grouping of tags. It will show up in landmarks. It typically consists of an h tag (h1, h2 etc.) and the content related to that h tag.

WCAG does warn however about nesting a header or footer tag inside of a section or article tag.

If a header or footer is nested it will not get the WAI-ARIA banner role or have special behavior in Assistive Technologies (AT).

Article:

An article tag is similar to a section tag, however, is typically used to divide a page into subgroups. Think of one webpage with multiple blog posts. Each blog post will be wrapped in an article tag.

Aside:

An aside is usually a region on the webpage that can support the main content. They are separate and meaningful but are different. These could be related article lists or a definition for a topic listed in the main content.

Footer:

A footer is similar to a header. It usually contains information about the site, helpful links, or legal information.

HTML5 Page regions using WAI-ARIA

Web Accessibility Initiative's Accessible Rich Internet Applications Suite (WAI-ARIA) "defines a way to make Web content and Web applications more accessible to people with disabilities. It especially helps with dynamic content and advanced user interface controls developed with HTML, JavaScript, and related technologies." (Initiative, n.d.)

Using Aria roles on tags, you can improve usefulness of AT for disabled users.

For the following tags, you will be able to use the corresponding roles:

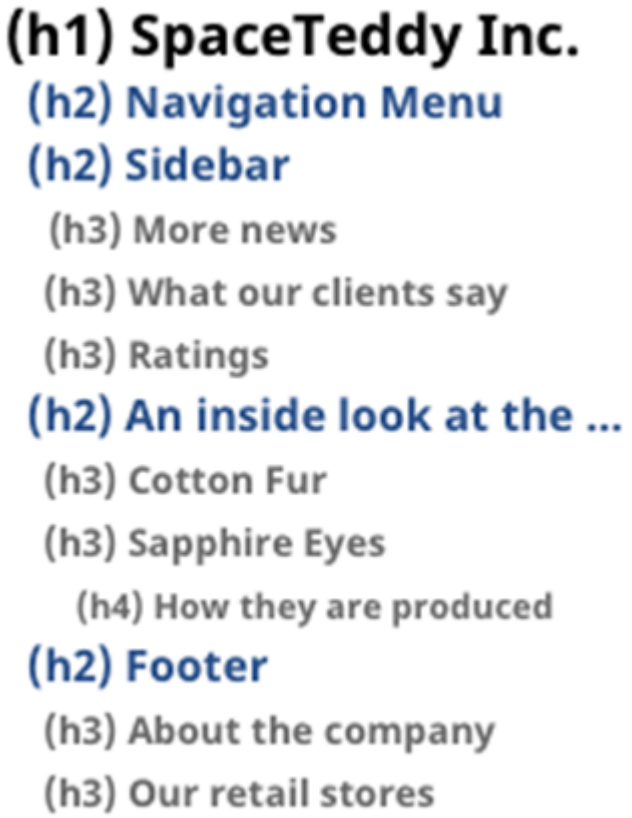
Headings

Another way of organizing your page is by using the heading tags (h1, h2 etc.). Headings are ways to structure content on the page, but it is also a major navigation element for screen readers.

Headings go in a hierarchical pattern starting with h1 being the largest to h6 being the smallest.

Heading tags should go in order h2 being nested inside a h1 tag, not the other way around.

You can have heading tags of the same value as siblings or on the same level in the hierarchy. A h2, should not be nested inside another h2.



*Src: **https://www.w3.org/WAI/tutorials/page-structure/headings/ (https://www.w3.org/WAI/tutorials/page-structure/headings/)***

If you are using Headings inside regions of the webpage, use the aria-labelledby attribute to set a label to the parent container.

For more information on how to **use headings refer to the WAI's tutorials (https://www.w3.org/WAI/tutorials/page-structure/headings/)** on them.

© Algonquin College 2022

Tables

Like figures and images, tables are a great way to convey data to a user.
Tables will typically be divided into data cells and table headers.

View the following table below for an example concert list from upcoming bands.

Bands	City	Date
Jimmy and the Jams	Toronto	July 16 2022
The Pork Sandwiches	Ottawa	August 31 2022

In the table You have three headers. Bands, City, and Date

The rest of the cells are strictly data cells.

To a user using a screen reader, you need to apply extra information to the table to allow for easier use.

Watch the video tutorial on how to apply the HTML code to table tags, you will need it for the next self check quiz.

- **https://www.youtube.com/watch?v=npto50RJpDc (https://www.youtube.com/watch?v=npto50RJpDc)**

There are many different types of data tables and each one requires their own instructions for accessibility improvement.

WAI has a tutorial breaking down different variations and standards for tables. (https://www.w3.org/WAI/tutorials/tables/)

© Algonquin College 2022

Form Instructions

Do forms need instructions?

Yes, instructions for forms are especially important. Instructions provide clear guidelines for filling out information and can help users avoid frustrations of filling forms out incorrectly.

An example of this is a “register” form for users to sign up. Usually a log in account needs a password. There are several formats of required passwords and typically they need at least 8 characters with a mixture of uppercase, lowercase, numbers, and special characters.
Imagine not having any clear guidelines on how to make your password prior to clicking submit. Not knowing whether the password you entered meets the requirements.

Instructions are even more important to screen reader users as some screen readers have a “Form Mode” which only reads out input, select, textarea, legend and label tags.

Form Instructions

If your form has certain instructions, such as time limits, needed personal information (like driver’s licenses) or other instructions that pertain to the entire form, then create an instructions list prior to the form tags.

These instructions will be read out prior to the screen reader entering “Form Mode”.

Example of instructions could be:

-Dates should be in the format dd/mm/yyyy

-All required fields must be completed

-usernames cannot have number characters in them

In-line Instructions

There may be certain form controls that need to have instructions with them to help users understand what is required of them.
To provide instruction you can use multiple approaches.

1) **Using labels:** You can provide instructions using an effective label tag. Example:
<label for=”password”>Enter Phone Number (xxx-xxx-xxxx): </label>

2) **Using aria-labelledby:** You can apply an aria label to a tag outside of the label and input tags. This provides a nicer visual design but is also accessible. Note: This is not dependable across all browsers and assistive technology.

See the example below

Example

Expiration date:
MM/YYYY

Code Snippet

```
<label id="expLabel" for="expire">Expiration date:</label>
<span>
  <input type="text" name="expire" id="expire" aria-
labelledby="expLabel expDesc">
  <span id="expDesc">MM/YYYY</span>
</span>
```

https://www.w3.org/WAI/tutorials/forms/instructions/#providing-instructions-within-labels (https://www.w3.org/WAI/tutorials/forms/instructions/#providing-instructions-within-labels)

3) Using aria-describedby: This approach is similar to the previous design however you use aria-describedby for the description rather than using the label. This will be announced in the screen reader after the label and input information.

See Image below for an example.

Example

Search: e.g. Apple Pie
Email: joe@example.com

Code Snippet

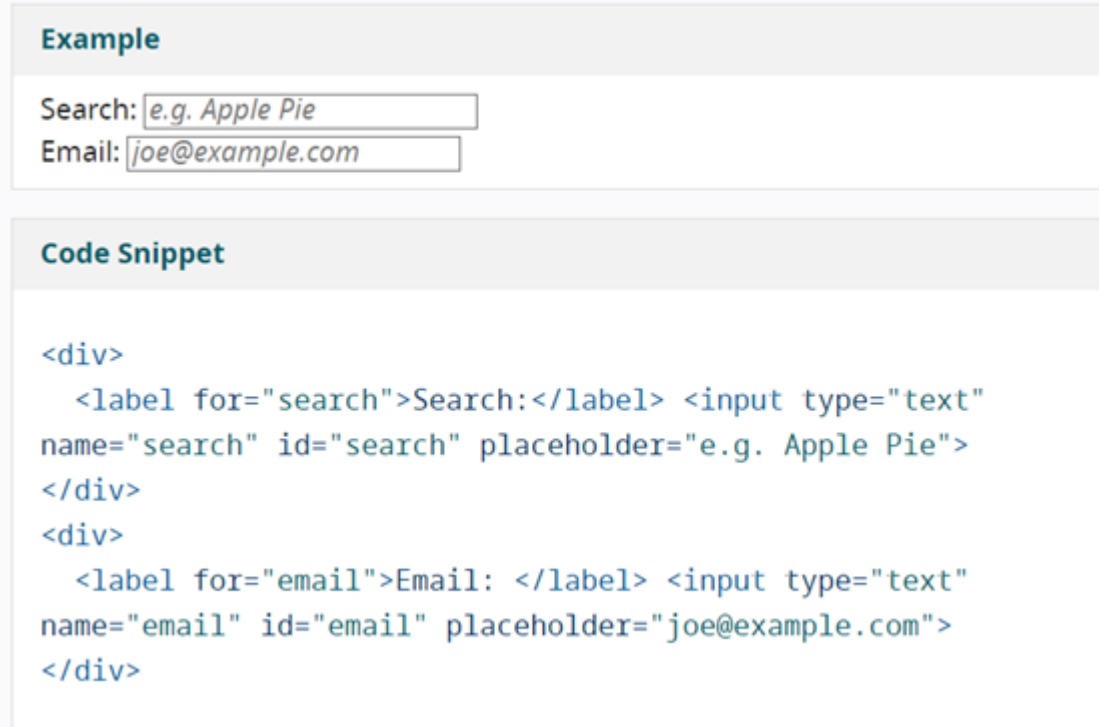
```
<div>
  <label for="search">Search:</label> <input type="text"
name="search" id="search" placeholder="e.g. Apple Pie">
</div>
<div>
  <label for="email">Email: </label> <input type="text"
name="email" id="email" placeholder="joe@example.com">
</div>
```

https://www.w3.org/WAI/tutorials/forms/instructions/#providing-instructions-within-labels (https://www.w3.org/WAI/tutorials/forms/instructions/#providing-instructions-within-labels)

4) **Using placeholder text:** You can provide placeholder text to show an example of what should be entered into a text field. Do note, if you are using this method, ensure you set the placeholder text to a high enough contrast as the default browser contrast does not meet WCAG standards. They are also not fully supported by all assistive technology or browsers.

Do Not use a placeholder as a replacement for labels. Placeholders should always be used in conjunction with labels.

See Image below for an example.



https://www.w3.org/WAI/tutorials/forms/instructions/#placeholder-text
(https://www.w3.org/WAI/tutorials/forms/instructions/#placeholder-text)

Finally for more information on instructions please visit the WCAG site.
(https://www.w3.org/WAI/tutorials/forms/instructions)

© Algonquin College 2022

Grouping controls

What is grouping?

Grouping in forms relates to making controls in a form like checkboxes or radio buttons into a group rather than having them visually together, but to screen reader users, separate.

It allows for users to focus on a smaller more manageable groups rather than a giant form.

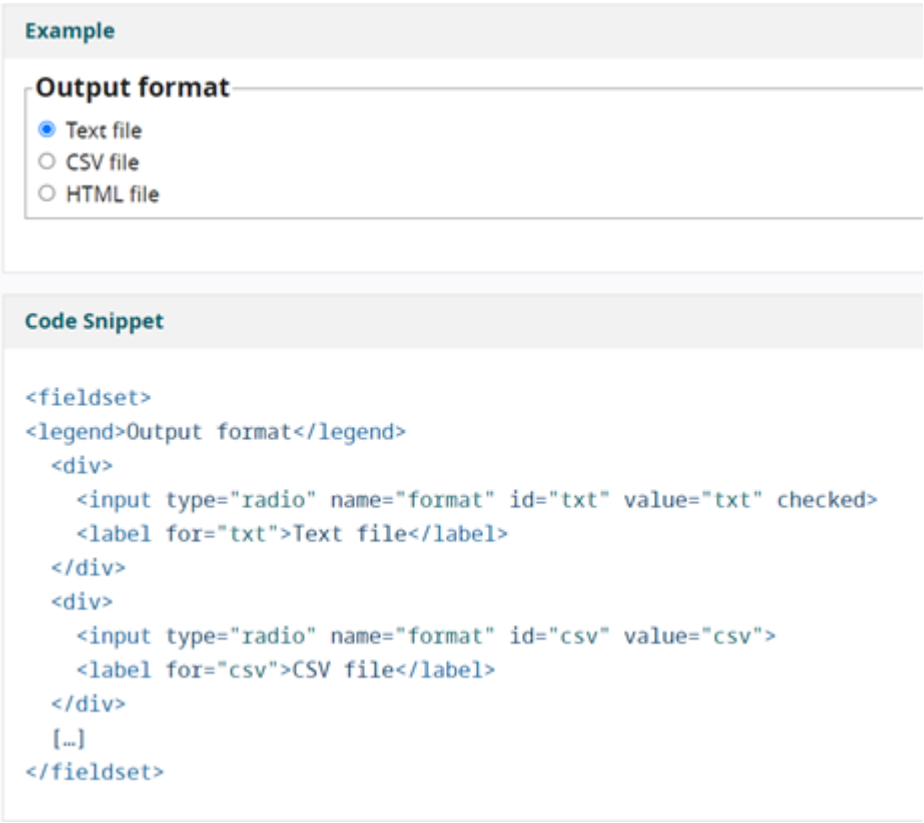
You can also use groups to group like fields together, for example grouping shipping address information then grouping billing address information.

Using the Fieldset tags

If you want to group elements together you are able to **use the <fieldset> tag (https://www.w3schools.com/tags/tag_fieldset.asp)** to apply a grouping to related elements on the form.

A fieldset tag can have nested inputs and label tags inside it.

If you want to apply a title or text to a fieldset group, you can use the >legend> tag. It is recommended to do this so screen readers can read out what this grouping is for.



https://www.w3.org/WAI/tutorials/forms/grouping/ (https://www.w3.org/WAI/tutorials/forms/grouping/)

For separate related fields you can apply multiple fieldsets in a form tag.

Code Snippet

```
<fieldset>
  <legend>Shipping Address:</legend>
  <div>
    <label for="shipping_name">
      <span class="visuallyhidden">Shipping </span>Name:
    </label><br>
    <input type="text" name="shipping_name" id="shipping_name">
  </div>
  <div>
    <label for="shipping_street">Street:</label><br>
    <input type="text" name="shipping_street" id="shipping_street">
  </div>
  […]
</fieldset>
<fieldset>
  <legend>Billing Address:</legend>
  <div>
    <label for="billing_name">
      <span class="visuallyhidden">Billing </span>Name:
    </label><br>
    <input type="text" name="billing_name" id="billing_name">
  </div>
  <div>
    <label for="billing_street">Street:</label><br>
    <input type="text" name="billing_street" id="billing_street">
  </div>
  […]
</fieldset>
```

https://www.w3.org/WAI/tutorials/forms/grouping/ (https://www.w3.org/WAI/tutorials/forms/grouping/)

Note: Sometimes a screen reader will read out a fieldset's legend for each nested input element or not at all, depending on the configuration. Try to use a concise legend when a screen reader reads every element and make each input label self explanatory when it does not read them.

Using WAI-ARIA for Grouping

The second option for grouping form elements is by using the Aria "role=group" and "aria-labelledby" attributes.

To apply this to your form set the role to group and apply the aira label to the element that has the text for the group. See the image below for how to code with aria grouping.

Finally, when grouping select and options elements you can use the <optgroup> tag to group the options available.
This is good for groups that have common themes.

See Image below for code use.

Code Snippet

```
<select>
  <optgroup label="8.01 Physics I: Classical Mechanics">
    <option value="8.01.1">Lecture 01: Powers of Ten</option>
    <option value="8.01.2">Lecture 02: 1D Kinematics</option>
    <option value="8.01.3">Lecture 03: Vectors</option>
  </optgroup>
  <optgroup label="8.02 Physics II: Electricity and Magnestism">
    <option value="8.02.1">Lecture 01: What holds our world together?</option>
    […]
  </optgroup>
  […]
</select>
```

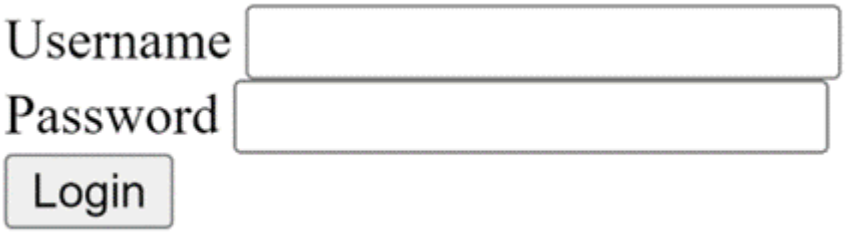
https://www.w3.org/WAI/tutorials/forms/grouping/ (https://www.w3.org/WAI/tutorials/forms/grouping/)

Remember that grouping helps organize the form fields for easier use both for screen reader and visual users.

© Algonquin College 2022

Labeling Controls

An HTML form is a way of gathering information from a user. You use them anytime you log into a website, comment on a video or social media post and whenever you fill out an order when buying things online.



Forms are created by **using a <form> tag (https://www.w3schools.com/tags/tag_form.asp)** to allow the HTML page to expect a form and to hold data about that form inside.

You then use <input> tags with <label> tags to let users enter data. The first images form was made using the HTML from the image below

```
<form>
    <label for="name">Username</label>
    <input id="name" name="username" type="text">
    <br>
    <label for="pw">Password</label>
    <input id="pw" name="password" type="password">
    <br>
    <input type="submit" value="Login">
</form>
```

There are various versions of the input tag's type such as radio buttons, checkboxes, text fields, and buttons. **Find out more on the structure and types of inputs here. (https://www.w3schools.com/tags/tag_input.asp)**

Finally, the focus on this module is how to accessibly design a form and to start, labels play a large part of making forms more accessible.

Labels

A label tag is as the name suggests, used to label a form input element.
Without a label your login form would look like so (Image7.3)

While you can forgo the label tags and just use text, it is not recommended to do so because it is not accessible.

When you assign a label to an input tag, a screen reader will say the text of the label and then when entering the input field it will say something like "Username edit text" for the username field. If a label is not attached, it would say something like: "Edit text blank" which is not helpful for screen reader users.

Ensure the for attribute in the label tag matches exactly to the corresponding input tag to allow for association.

If you do need to hide a label (for example a search field) there are 4 suggested methods from WAI.

The 4 approaches are as follows:

- Hiding the label element: This uses CSS to build a 1x1 pixel placement for the label and using overflow:hidden to make the label not appear visually but is still technically there for screen readers.
- Using aria-label: Rather than using a label element, you apply an aria-label to the input field.
- Using aria-labelledby: Similar to the above method, however you are labeling it using another element (usually the button)
- Using the title attribute: Applies a title to the input tag. This method is generally not recommended.

For **more information and code samples on hiding label elements (https://www.w3.org/WAI/tutorials/forms/labels/#hiding-label-text)** visit this site.

There is also a method for implicit labeling if explicitly (using “for” and the input id) cannot be done. To apply an implicit label, you take the input element and nest it in the corresponding label tag.

When labeling buttons you typically will use the “value” attribute for “input” type buttons for screen readers.

<input type=”submit” value=”submit”>

Normal HTML button tags have their value in between the opening and closing tags.
<button type=”submit”>Submit</button>

If you are using an image (like magnifying glass search button) for a button, then your label is set in the alt attribute.

<input type=”image” Source=”mglass.png” alt=”Search”>

Finally, labels generally go from left to right,

Username _input field___

Or occasionally above if possible, designing for mobile.

Username

_input field___

Just use common sense for your labeling and aim for accessible use.

© Algonquin College 2022

Forms are used everyday to log into websites, searching on google or adding data to a website. Since forms are so important to day-to-day online use, making them accessible is imperative.

In this module you will learn best practices for making accessible forms.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Create useful form feedback for users when validating input

02 Provide clear labels and instructions for form controls

03 Group like form controls to provide a clearer form

Key Terms and Concepts

1. AJAX: Asynchronous JavaScript and XML. A way of sending a request to a server without leaving your webpage
2. Dialog: A popup window on a browser with information contained in it
3. Form Control: a form input field and label

User Notification

Feedback

Imagine the following scenario: You are trying to login to your email account. You have entered an email and password then clicked the “Login” button, but nothing happens.

What is wrong?

Well to you the user there could be numerous problems that could be happening, your email could be incorrect, your password could be incorrect, the website could be down, the login button could be coded incorrectly or perhaps there is just something else.

The bottom line is a bad user experience as you are frustrated that you cannot log into your account.

Part of being a developer is to ensure you create a welcoming helpful experience for your users and that starts with feedback. Providing a simple message saying “email or password is not correct” let us the user know that they need to check those fields as they may have mistyped an entry.

Creating Feedback

The goal of feedback is to let the user know what is incorrect with their submission so they can fix it. There are several methods to provide feedback after a user submits their form.

1. Using the main heading
2. Using the Title page
3. Using Dialogs
4. Listing Errors

Using the main heading

You can set the main heading on the form page to show any issue that appeared using the <h1> or <h2>. This allows for big warnings for low-vision users and a heading for non-visual users.

<h1>2 Errors – Credit Card Info </h1>

Ensure you list how many errors to let the user know how many fixes they need to complete. It is annoying for users to fix one error only to be confronted by the next.

Using the Title page

This example is especially useful for screen reader users as the title tag is one of the first things announced on a screen reader. The user will know right away that there is an error that has occurred and that they will have to fix it.

<title>2 Errors – Credit Card Info </title>

Using Dialogs

Using dialogs for error messages are helpful since they are obtrusive and distracting. You can use a **JavaScript alert function (https://www.w3schools.com/jsref/met_win_alert.asp)** to create this dialog box. This dialog matches the user's font settings so if they have custom fonts the dialog will match. If coding a custom dialog, ensure you match the fonts of the user.

For a visual example and a sample of the code visit WAI's dialog example. (https://www.w3.org/WAI/tutorials/forms/notifications/#using-dialogs)

Listing Errors

It is extremely helpful if a submission is unsuccessful to list all the errors found on the form above the form. The errors should include:

- Which part form control/label the error occurred on
- A description of the so the user can understand why it happened
- A suggestion on how to fix it, what format pattern the field is expecting
- A link to jump to that spot in the form for quick access.

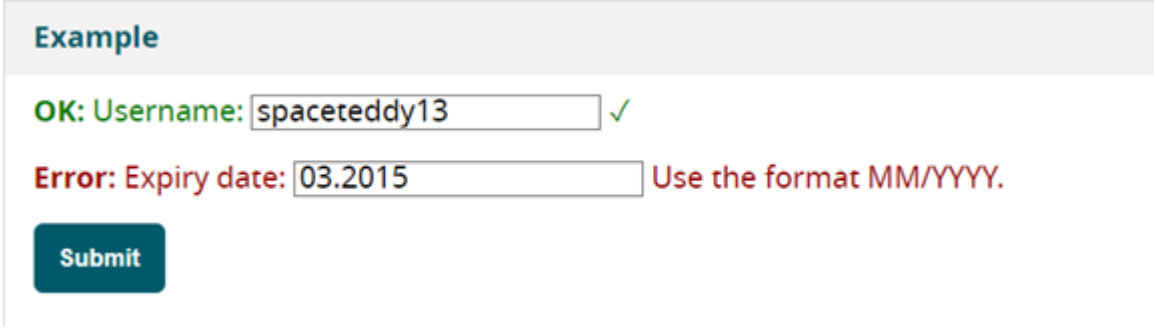
If you are using AJAX calls, then dynamically show the list of errors at the top of the form in a container. Set the role attribute to alert so assistive technology can be aware of the changes.

For a visual example and a sample of the code visit WAI's dialog example. (https://www.w3.org/WAI/tutorials/forms/notifications/#listing-errors)

In-line Feedback

Instant feedback is important for users to understand what is happening on the form. To accommodate this, use in-line feedback on the same form control the user is currently on.

Using visual cues and feedback go a long way to help the user easily identify whether they have successfully filled out that section or not. Visual cues like a green boarder or checkmark indicate success whereas a red border and feedback instructions indicate that there are still fixes that need to be completed.



There are three approaches on when to give in-line feedback:

1. After submission
2. During typing
3. On focus change

After Submission

You can provide in-line feedback when after a user submits a form. This usually involves some JavaScript to change classes/elements programmatically when the button is pressed.

For a visual example and a sample of the code visit WAI's dialog example. (https://www.w3.org/WAI/tutorials/forms/notifications/#after-submit)

During Typing

This type of feedback is instant and changes as the user types. This is helpful for the user to get immediate feedback without having to submit.

For this feedback you utilize the **JavaScript "key up" event listener. (https://www.w3schools.com/jsref/event_onkeyup.asp)**

Essentially when the user lifts their finger after typing, that triggers an event that JavaScript can then listen to. This gives JavaScript a moment to react to the user and by reading that input field it can do a quick validation and respond with feedback instantly.

While some client-side validation can be done (like pattern checking or not null checking) some cases may need server-side validation (like username availability). If you need any server-side validation you can do so by using AJAX calls, however these calls take time and are best used for focus change feedback.

You can provide two sets of feedback during typing the first is binary messages.
This feedback has two states, without message or with message. The message does not change when shown.

For example, a required field when empty shows nothing. When it is filled you add a green checkmark. That gives it two states.

The second type of feedback has scaled feedback. This is useful for things like passwords that scale based on the complexity of a password from a weak to strong password and can provide that feedback immediately.

On Focus Change

Finally the last type of feedback happens when you shift focus from a field. This is known as **JavaScript's "focus out" event (https://www.w3schools.com/jsref/event_onfocusout.asp)**. This happens when you tab off of an input or leave that input element.

This is useful when users are entering data in a specific format (like date) or when using AJAX calls to check availability of something (like usernames)

Aria-Live

What is Aria-Live? (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Live_Regions)

You can apply the aria-live attribute to certain HTML elements that can contain your feedback messages like span tags.

An Aria-live dictates how the message gets detected and conveyed with a screen reader. There are three values, off, polite, or assertive.

"Off" sets it to ignore the message. "Polite" will notify the user once the user has been idle, just like a friendly reminder.

Finally there is "assertive". Assertive will immediately interrupt the user with the notifications. This should be used when it is a time sensitive or critical notification.

Review

For a quick review of this module please watch this short 5-minute video

- **https://www.youtube.com/watch?v=72nrJJAf_Ak (https://www.youtube.com/watch?v=72nrJJAf_Ak)**

© Algonquin College 2022

Validating Input

Validation

Mistakes happen all the time. It is common practice to validate user input in both the front and back end of a webpage. In this section we will focus on front end validation and feedback.

Validation in terms of forms can be done with HTML tag attributes and with JavaScript.

Required Input

If you have a form field that MUST be completed, then it is a “Required” field.
In HTML5 if a field is required a user will not be able to submit the form until that field is filled. It will also produce a popup saying something like “Please fill out this field”

When building a required field there must be 3 things needed.

1. A label that notes that the field is a required field. This is good design as it is clear to both visual and non-visual user that it is a required field without having to do trial and error of submitting and getting an error message.

2. Adding a “required” attribute. This attribute sets the field to have to be completed before submission.

3. Adding aria-required: The aria-required attribute allows assistive technology to know that this is a required field. Note: most browsers set this by default to true if an HTML5 required attribute is set.

Together those three look like the following:

<label for=”password”>Password (required): </label>

<input type=”Text” name=”password” id=”password” required aria-required=”true”>

Common Input Types

In HTML6 there are input types that are commonly used that have some validation built in.

These include email, URL, number, range, date, and time.

- **Email: (https://www.w3schools.com/tags/att_input_type_email.asp)** email has a quick pattern check to see if there is any character then an @ finally another character. (**Ex. name@email.com (mailto:name@email.com)** and T@L would be valid emails to that field)

- **URL (https://www.w3schools.com/tags/att_input_type_URL.asp)**: A URL type matches text that starts with http:// or https://

- **Number: (https://www.w3schools.com/tags/att_input_type_number.asp)** As the name suggests it will take a number. You can set a min and max attribute to limit the acceptable number's range

- **Range: (https://www.w3schools.com/tags/att_input_type_range.asp)** Range like number takes a numbered range between it's min and max attributes. The difference is rather than text input it takes a slider for input.

- **Date: (https://www.w3schools.com/tags/att_input_type_date.asp)** Date allows for a drop-down calendar to allow users to choose a date from it. The modal popup window allows for keyboard and tab controls. Date is in yyyy-mm-dd format.

- **Time: (https://www.w3schools.com/tags/att_input_type_time.asp)**Time allows for a drop-down window where a user can the time they need. The format is HH:MM pm/am.

Pattern Validation (https://www.w3schools.com/tags/att_input_pattern.asp)

HTML5 has an attribute called "pattern" lets you apply a **regular expression (https://en.wikipedia.org/wiki/Regular_expression)** to set a custom validation pattern to filter good input.

An example of an input validation with pattern is as follows:

<input type="text" name="province" pattern="[a-zA-Z]{2}" title="Two letter province code">

The pattern all depends on how you expect input to be received. Make sure you set an instruction on how to enter the input.

Final notes:

Make sure that you are forgiving in your input. Try to accept different input, for example US Zip codes are numbers only, whereas Canadian postal codes are alpha-numeric.

Try to use client-side validation when possible as it reduces network and server load, however, also perform a validation check on the server side as client-side validation can be bypassed.

Ensure users are able to alter their incorrect fields rather than just removing them. Allowing for the form to be auto populated with previous input prevents having users filling out the form multiple times.

Always try to have a summary page that shows the information to the user before submitting. This allows them to catch any issues with a potential order or any incorrect fields they made.

Provide user confirmation on any irreversible actions. Such as account deletion or bank transfers to name a few.

Finally, allow users to undo actions. For example canceling a purchase, undo sending an email or preventing a delete.

For more information on accessible validation visit the WCAG site. (https://www.w3.org/WAI/tutorials/forms/validation/)

© Algonquin College 2022

Colour Contrast

Contrast plays a big part in visual design for users with low-vision or colour blindness. When developing your web page, ensure you use a sufficient contrast text with the background colors. This will include any text on images, buttons, background gradients, or other elements.

With logos or text that may appear in a photograph (that's focus is not the text) you can ignore the contrast.

An example of good and bad contrast is as follows:

✗ Poor	✓ OK
If the text has low contrast between the text and background, it may be hard to read. For others you may need to use low luminance to if High luminance are not readable for them.	If the text has low contrast between the text and background, it may be hard to read. For others you may need to use low luminance too if High luminance (bright colours) are not readable for them.

Having good colour contrast complies with the **WCAG 1.4.3 Contrast guidelines (https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html)** of having a minimum contrast ratio of 4:5:1.

For an example of how **colour blindness affects users please read this user's story about online shopping. (https://www.w3.org/WAI/people-use-web/user-stories/#shopper)**

For checking and testing your contrast levels, visit the Web Accessibility Initiative (WAI) website (https://www.w3.org/WAI/test-evaluate/preliminary/#contrast)

These changes to your contrast can all be made through CSS. For testing examples please view Vid. 8.1

© Algonquin College 2022

Images and graphics

A picture is worth a thousand words. Sometimes having an image describe a concept or portray a feeling is better than describing it. Unfortunately, a picture is not enough for some users.

When implementing an image, diagram or graphic in order to comply with the **WCAG non-text content guideline 1.1.1 you must add alternative text describing that image (https://www.w3.org/WAI/WCAG21/quickref/#non-text-content)**.

Alternative, text helps users with low or no vision understand why the picture is there and what it does to help understand/compliment the content.

Alternative, text helps users with low or no vision understand why the picture is there and what it does to help understand/compliment the content.

When adding images to your site, **refer to the WCAG tutorials on how to implement them correctly. (https://www.w3.org/WAI/tutorials/images/)**

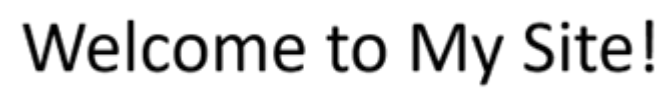
If using image carousels, add clear visual controls to allow users to stop any animations or auto playing sounds.

For instructions on how to create a proper WCAG compliant carousel visit the following link. (https://www.w3.org/WAI/tutorials/carousels/)

Adding both graphical icons in conjunction with your text can help improve both visually and accessibly your pages design. The icons can give a quick explanation of what the text refers to, and having that text helps users understand what the icons represent for nonvisual users.

Ensure that you use text rather than an image of text.

As an example, View the image below



It is a nice welcome image of text.

Compare it to the following text, generated in the browser.

“Welcome to My Site!”

To show why you do not use images of text, try to zoom in using the browser’s zoom. What you should see is that the image text gets blurrier the larger it is zoomed on, while the generated text doesn’t blur.

This is because generated fonts use vectors to generate them, and scale based on pixels. This also saves bandwidth and the text generated is smaller in memory than an image of text.

Finally, ensure if you are adding the graphics to your website that the contrast and colours of the graphic is viewable by users with colour blindness. Ensure captions and description are provided for graphics, tables, or graphs.

© Algonquin College 2022

Interaction Design

When designing interactive elements, you must ensure that they are easy to see, navigate to and easy to identify what it is.

For links, ensure they appear different when a mouse hovers over them, when a keyboard focuses on them and when a touch screen user is touching it. Ensure that your style for links is consistent throughout the website, for example if one page uses magenta links on a white background, and another page's background is white, don't change your link colours to something else.

When focus is on an element, ensure you highlight that element with a box or some other visual cue to show the user that they have selected that element.

Clear interactive elements and consistent elements comply with **WCAG 2.4.7 (https://www.w3.org/WAI/WCAG21/quickref/#focus-visible)** and **3.2.4 guidelines (https://www.w3.org/WAI/WCAG21/quickref/#consistent-identification)** respectively.

Part of interactive design lets users have clear and consistent navigation options. This is achieved by maintaining the same naming, styling and positioning conventions to comply with **WCAG 3.2.3 guidelines (https://www.w3.org/WAI/WCAG21/quickref/#consistent-navigation)**. Users must have multiple ways of navigating a site, this can include a site map or a breadcrumb trail of links to comply with **WCAG 2.4.5 standards (https://www.w3.org/WAI/WCAG21/quickref/#multiple-ways)**.



https://www.w3.org/WAI/tips/designing/ (https://www.w3.org/WAI/tips/designing/)

This is a bit more advanced, but a search bar to find topics in your site is a big help for users. **(https://www.w3schools.com/howto/howto_css_breadcrumbs.asp)**

An example of a breadcrumb trail can look like the following example from W3 schools. (https://www.w3schools.com/howto/howto_css_breadcrumbs.asp)

For some tips on developing accessible breadcrumbs view this website. (https://www.aditus.io/patterns/breadcrumbs/)

As mentioned in the last module, providing vibrant and thorough feedback on forms helps users understand what is wrong when they try to interact with that form.

Finally, try to avoid any “timed” interactions on your webpage. Some users with limited motor controls take longer to perform input and interact with websites. Ensure enough time is given to those users if time-sensitive content is on your website.

© Algonquin College 2022

Ensuring a website looks good visually helps visual users interact with your website in a positive manner, however visuals can mean a whole lot more to user with disabilities to help them engage in your content.

By adjusting certain elements of your webpage, such as colour, symbols, images, and videos you can help improve a non-visual user's experience on your website.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Improve a websites use of colour by adjusting the contrast ratio and adding text to colour only diagrams

02 Provide necessary alternative text and transcripts to multimedia elements on a webpage

03 Enhance interactive elements and responsiveness on a website to allow users with disabilities a more engaging experience

Key Terms and Concepts

1. Colour Contrast: The visible difference between colours on a website, typically text and background colours.
2. Responsiveness: Allows a webpage to be viewed on different media platforms and maintain readability and useability.
3. Module Material: Organically written learning material developed by a Subject Matter Expert.

Multimedia and animations

Most of the guidelines that work with images should also be applied to multimedia and animations. When possible, provide any transcripts for videos or audio. If you also have descriptive transcripts of any videos that goes a long way to improve accessibility.

Have clear controls presented for users to allow them to adjust playback speed, volume and any other controls that are needed for websites.

Use video players where close captions can be used for users who have limited hearing and finally if you have videos, allow for timestamps to allow users to easily find sections of the video (especially important on longer videos).

The WAI site has several suggestions for improving media on your website. (https://www.w3.org/WAI/media/av/)

© Algonquin College 2022

Responsiveness

Responsiveness in web design refers to how a webpage acts on different viewing platforms.

What this means is when a webpage is viewed on a desktop, developers use the extra space to allow for easier navigation and view of the content.

However when a webpage is viewed on a smaller device such as a phone or tablet, the webpage responds by increasing font size, and re-arranging the webpage's layout.

The images below show Algonquin College Online's website in both desktop and mobile respectively.



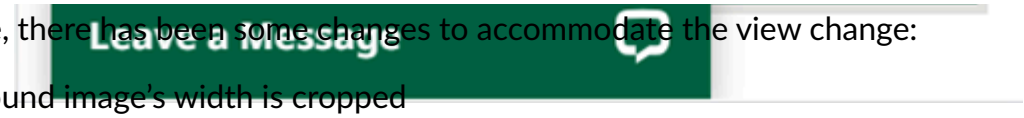
ALGONQUIN COLLEGE



Find Your Perfect Fit Online

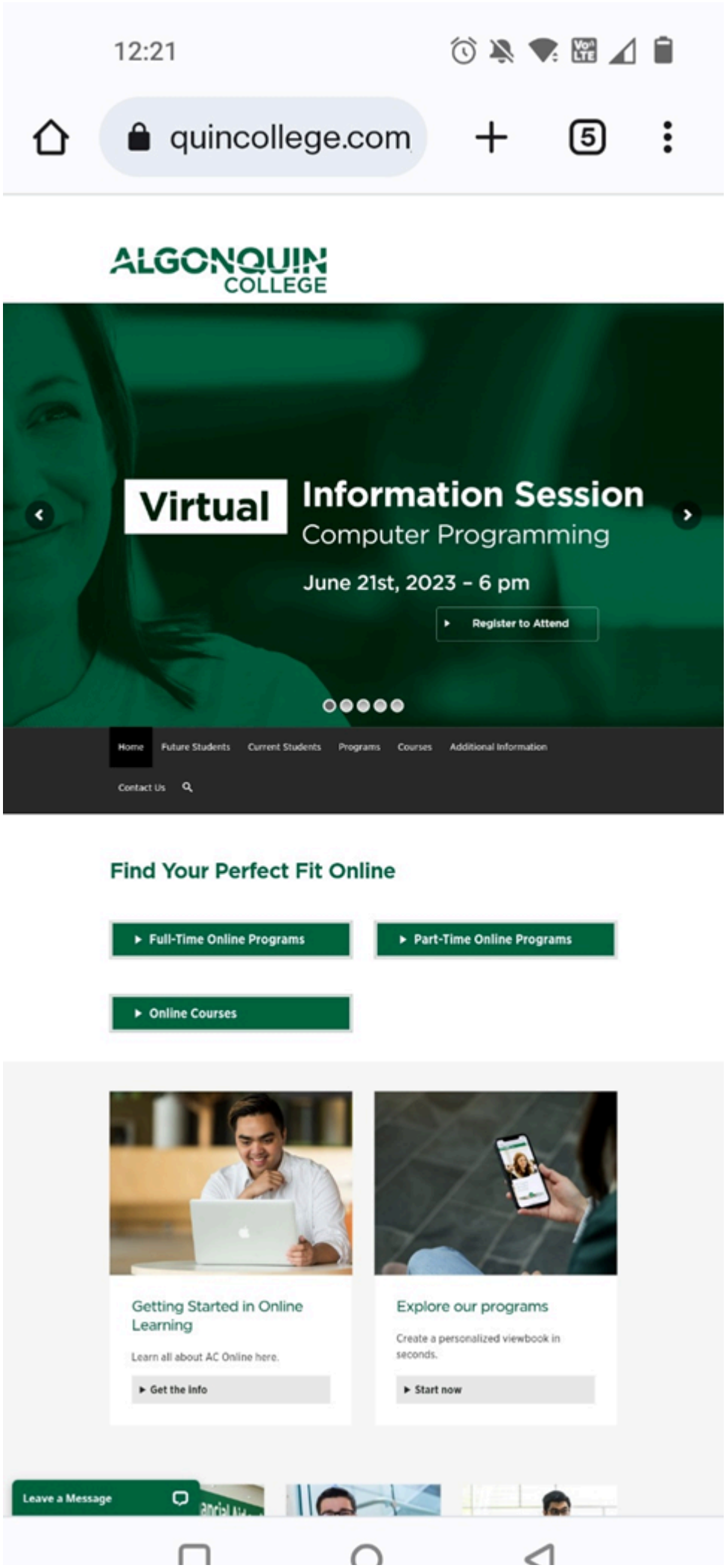
- Full-Time Online Programs
- Part-Time Online Programs
- Online Courses

If you notice, there has been some changes to accommodate the view change:



- Background image's width is cropped
- The navigation menu has been reduced to a drop-down hamburger menu
- The school information like campuses have been grouped into a drop-down in the "AC" Logo.

If there was no responsive design the website would not look great. The image below is the course website viewed on a mobile device with the "Desktop Site" mode set on the browser.


12:21
quincollege.com
5
ALGONQUIN COLLEGE
Virtual
Information Session
Computer Programming
June 21st, 2023 – 6 pm
Register to Attend
Home
Future Students
Current Students
Programs
Courses
Additional Information
Contact Us
Find Your Perfect Fit Online
Full-Time Online Programs
Part-Time Online Programs
Online Courses
Getting Started in Online Learning
Learn all about AC Online here.
Get the info
Explore our programs
Create a personalized viewbook in seconds.
Start now
Leave a Message

As you can see, its not great. The navigation items are hard to read since their font size is so small increasing the difficulty for low vision users, and all the clickable icons are much smaller making it harder for users with low motor skills to interact with the website.

When planning responsive design there are a couple things to keep in mind.

1. Increase the text size on mobile devices. Ensure your page is readable and functional when zoomed to 200% to comply with **WCAGs 1.4.4 guideline (https://www.w3.org/TR/WCAG21/#resize-text)**
2. Narrow your pages columns to one single main column for content.
3. Navigation and secondary information are revealed using an icon.
4. Increase the size of icons for visibility and interaction
5. Images in a row should be flexed into a column or carousel viewer
6. Ensure your text fields do not hide overflow when text size is increased

Web.dev has an article diving into specifics of responsive design which can be summarized by the following points:

“-Use the viewport meta tag

-Allow users to zoom

-Design with flexibility

-Use relative units for text

-Avoid disconnecting the visual view from the source order

-Take care with spatial clues

-Ensure tap targets are large enough on touchscreen devices”
(Accessible responsive design.)

More information:

Developing for a small screen size (https://www.w3.org/TR/mobile-accessibility-mapping/#h-small-screen-size)

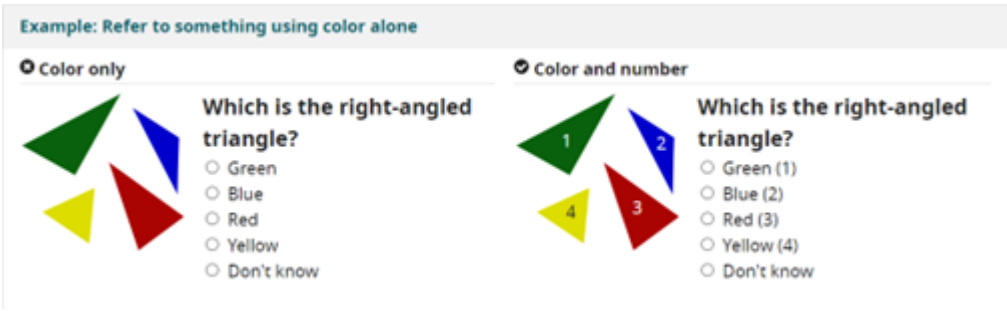
Creating understandable mobile pages that are WCAG compliant (https://www.w3.org/TR/mobile-accessibility-mapping/#mobile-accessibility-considerations-related-primarily-to-principle-3-understandable)

© Algonquin College 2022

Use of Colour

Colour can be a great way of adding nice visualization and to differentiate parts of a website. However, when designing features of a webpage it is imperative that you do not rely solely on colour.

Using symbols or numbers in conjunction with colour helps users who are colour blind to access your website. For example view the image which is from the Web Accessibility Initiatives (WAI) website.



https://www.w3.org/WAI/tips/designing/ (https://www.w3.org/WAI/tips/designing/)

It depicts two sets of designs, one using colour only and one using colour and numbers.

Running that image through a “green-blind” filter turns the image into below

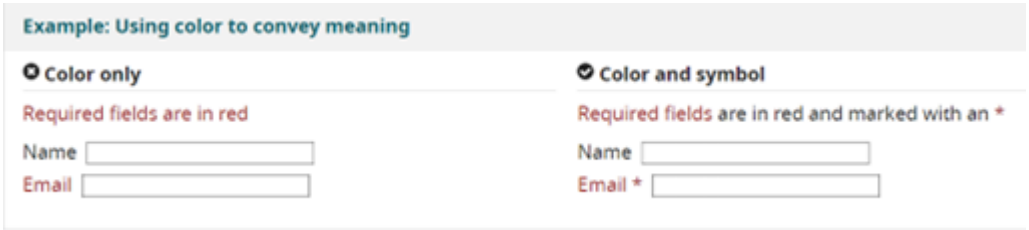


As you can see, the first version is not fair for green blind users to answer since both the green and red triangles have the same colour hue.

With the diagrams on the right of each image, both are easily answered by both non colour blind and colour-blind users.

Also take note that the yellow triangle uses correct font contrast (black text rather than white).

Another example from the WAI site shows form instructions for required fields.



https://www.w3.org/WAI/tips/designing/ (https://www.w3.org/WAI/tips/designing/)

As you can see, the first example only uses colour to differentiate required fields.

After running it through a “red-blind” filter the image turns into Img. 8.5.

While there is a very slight contrast difference, having the asterisks is a very quick way for colourblind users to view the required fields.

For a summary of colour use and colour contrast watch the videio below:
https://www.youtube.com/watch?v=BW_PjPR825w (https://www.youtube.com/watch?v=BW_PjPR825w)

© Algonquin College 2022

CSS, styling & hidden content (skipping page content)

There may be times when you want to hide content. This may be because you want to have a toggle that reveals more information to the user.
If this is the case and you want it hidden both from visual and non-visual users, you would use a CSS property or HTML attribute to hide it.

The CSS properties to hide them are:

display:none
or

visibility:hidden

The HTML attribute “hidden” works the same way, but is not supported by older browsers.

Finally the last way to completely hide the element is to use width or height at 0px.
While these all remove elements from view, they do not get rendered to the DOM. Ensure that you do this only when needed.

Only Visually hidden

There are certain times when you may want to hide elements visually, but not to screen readers.

These could be special instructions for screen reader users or specific information about a visual only element.

While you should most time have content present for both demographics of users, there may be certain cases where you need those visually hidden elements.

To accomplish this, you will use CSS to position the elements far off outside of the screen’s viewing are.

The following CSS shows how to apply this:

Here is a breakdown of what the code above means:

.sr-only The CSS will only apply to this class. Sr-only = Screen Reader Only

position:absolute Let’s the browser know that this element removes itself from the regular flow of the webpage’s DOM.

left:-10000px Moves the element -10000 px to the left.

top:auto; Sets the vertical position to the top of the page.

width:1px, height:1px Sets the width and height to a single pixel.

overflow:hidden Sets the container to hide any part of the element that doesn’t fit in the

1x1 pixel container. Essentially hides it all.

Finally to apply this to any element I’m hidden .

Try not to hide any links or interactive elements since visual users using a keyboard can navigate to those links but not be able to see them.

For more information on hiding content visit the Webaim’s website. (https://webaim.org/techniques/css/invisiblecontent/)

© Algonquin College 2022

Reading Order

Getting to the Content

When a user engages with a website, they do so for the content on the website itself.
Content is any images, text or media present on the website. When building content for users with accessibility you have to keep in mind how the user will “view” the content.

As previously discussed, ensure you have correct colour contrast that complies with WCAG standards.

Another consideration is the order in which a screen reader reads.
Typically, a screen reader will read from left to right, top to bottom based on the elements in a Document Object Model (DOM). When building a website ensure you have this focus planned out for a screen reader to read from.

A screen reader will start with the <title> tag, this should give information about the page itself ex. “Algonquin College Online | Current Students”, this immediately tells the user what the page is and what website it is on.

The next announcement will usually be the URL. If the title is not adequate than the user can confirm their site using the URL address.

After the URL is a hidden “skip to content” link. This allows the user quick navigation to the part that they want to access.

After the skip link, the screen reader will go through the lists of Navigation links and banners.

Finally, it gets to the content.

The image below shows the order of navigation and how a screen reader will read each section.

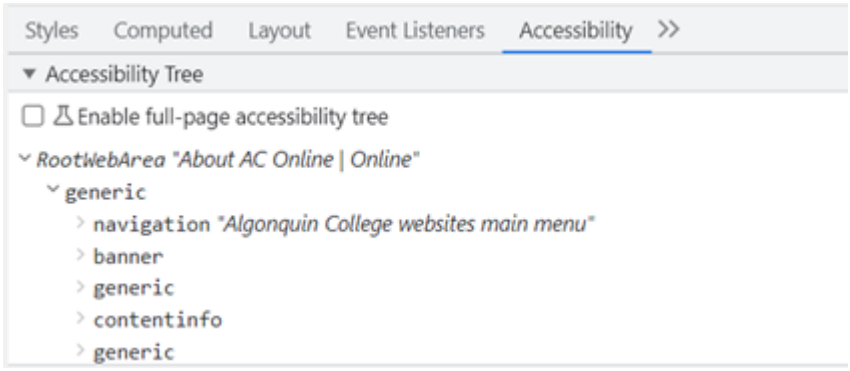


The following image has a legend of what it reads.

Section #	Description
1	Title Tag
2	URL Address
3	Skip content Link
4	Nav links Top
5	Banner and Video
6	Nav links below
7	Main Content
8	Footer and other modules (E.g. AC contact Chat)

Content Layout

With content, the screen reader will work its way from the top of the accessibility tree downwards. See Image below.



From there It will typically start with headers then the following paragraphs.

The page structure should be similar to how you would lay out a normal document. One consideration when building content is how things are laid out in HTML.

When laying out content, imagine you are designing a layout like Image 9.4.
It has 2 sections of text and quick navigation on the right.
While, to us visual readers, we would read the First and second sections then proceed to the Links. However, since the actual structure grouping them shows in Image 9.5, a screen reader wouldn't read it in the same order.

The screen reader would read the first section, the links then the second section.

This is because the structure is setup that way. While visually it looks nice, structurally it doesn't work well.

A better solution is to have an outer div, containing all of them, then the left-hand grouping would have it's own div and the right hand would have it's own as well.

From a structure like this table:

1.	2.
3.	

To a structure like this table:

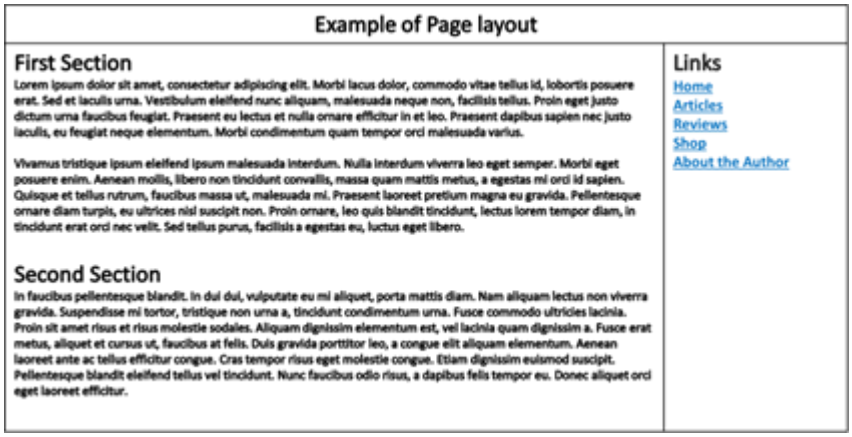
1. 2. –	3.

Since the left-hand grouping is in the first column, they would have a higher priority than the right column to screen readers.

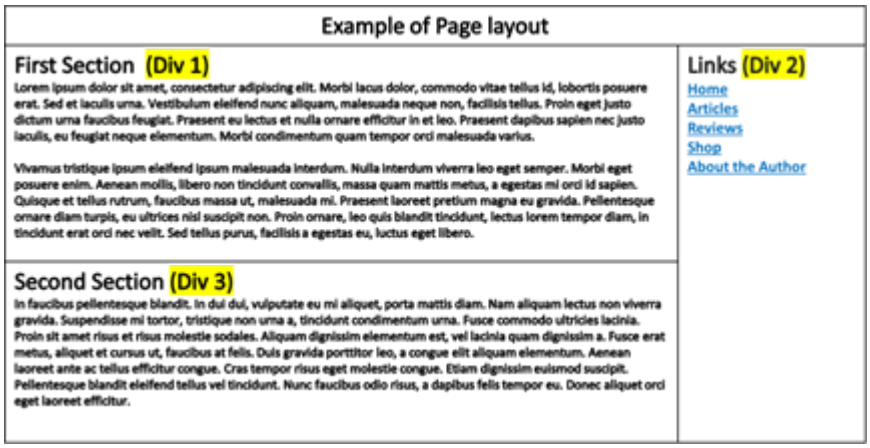
Here is an example of a screen reader reading a document poorly

- **https://www.youtube.com/watch?v=QmPr-1-HnUU (https://www.youtube.com/watch?v=QmPr-1-HnUU)**

Let the HTML Dictate where your content on the webpage will be. CSS should compliment that structure, not dictate it.
A very poor example of CSS determining the layout over structure can be found in the image below



In this example, the code on the left has 2 sections the div class first, and the div class second.



The screen reader will read the HTML code in this order, however, there has been CSS styling to change the visual order of theses two elements. This causes confusion for the correct order of these sections to be read.

In short, remember to let HTML structure your webpage, and ensure your structure lets the screen reader correctly read your content in the order you wanted.

Another great visualization of reading order can be seen in this video: **https://www.youtube.com/watch?v=8MAvH6vYbDo (https://www.youtube.com/watch?v=8MAvH6vYbDo)**

© Algonquin College 2022

Writing Accessible Content

Title

When writing content the title and headings help non-visual users quickly know if they are on the correct page.

When creating an accessible title, ensure it has a short clear text that answers the following questions:

- What site is this?
- What section of the site is this?
- If this is a multi-step process, which step is it on currently?

Example:

The website is Algonquin College Accessible Committee.

A good title for the home page: “Algonquin College Accessible Committee”

It specifies where you are.

Now if you navigate to the about page: **“About – AC Accessible Committee”**

What about a signup form: “**Sign up (Step 2 of 3) – AC Accessible Committee”**

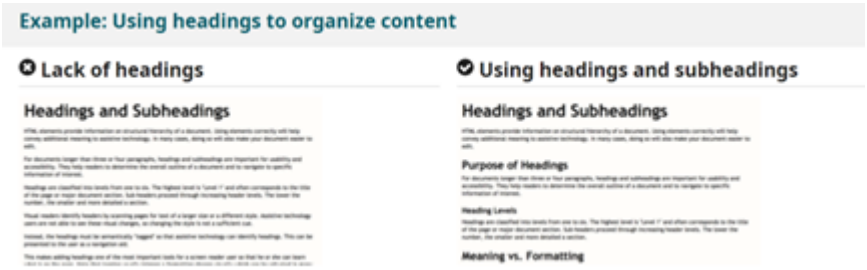
Headings

The goal of headings is twofold:

1. Have navigation elements for screen readers
2. Easily breakup content into understandable sections

You have learned already that if using semantic HTML for creating content that you will naturally be creating landmark elements for screen-reader users.
Secondly, headings help break long text into easy to digest section for both screen reader users but also visual users.

The Image below shows two content blocks. One with a lack of headings and one with headings and subheadings.



https://www.w3.org/WAI/tips/writing/ (https://www.w3.org/WAI/tips/writing/)

The latter helps you to easily find the topics you need without having to read through a big block of text.

Links

A screen reader will read the text of a link, so having clear concise values of an <a> tag provide screen reader users the ability to quickly understand where that link will take them.

E.g. To sign up for the Algonquin College Accessible Committee, click here(This will only read “Click Here”)

Change it to:

To sign up for the Algonquin College Accessible Committee, click here (This will read the whole sentence, making it both easier to understand for screen reader user, but also make the link have a larger click area for users with low motor control.)

Images

For images ensure your text alternatives describe the image, not just what the image is. For example, if you have any diagrams like tightening a screw with a screwdriver.
Instead of the alt text being “a Screw and screwdriver” describe what the diagram is showing, “insert the screwdriver into the screw head and turn clockwise to tighten”

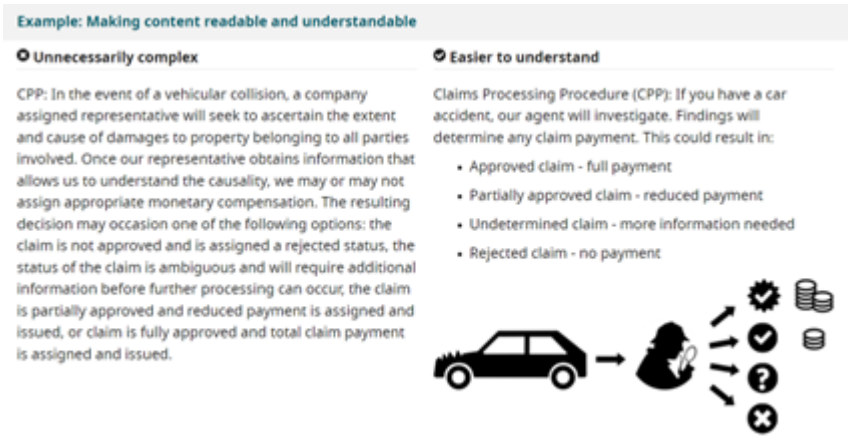
Clear and Concise

The most important thing to do is keep your content clear and concise. This means your content should be:

- Broken into small, clear sentences and paragraphs
- Try to keep your language simple and clear. Avoid jargon unless necessary
- Explain the acronyms when you first use them, then feel free to shorten it to the acronym when referring to it later. E.g. Web Accessibility Initiative (WAI).
- If there is specific jargon or term used, provide a glossary to look up their meanings.
  -Break steps or grouped information into lists when possible

- Use graphics, images, video, audio and symbols with text to convey meaning.

An example of good and bad content can be found in the Image below



https://www.w3.org/WAI/tips/writing/#keep-content-clear-and-concise (https://www.w3.org/WAI/tips/writing/#keep-content-clear-and-concise)

A story about a student having a difficult time reading content can be found here. (https://www.w3.org/WAI/people-use-web/user-stories/#classroomstudent)

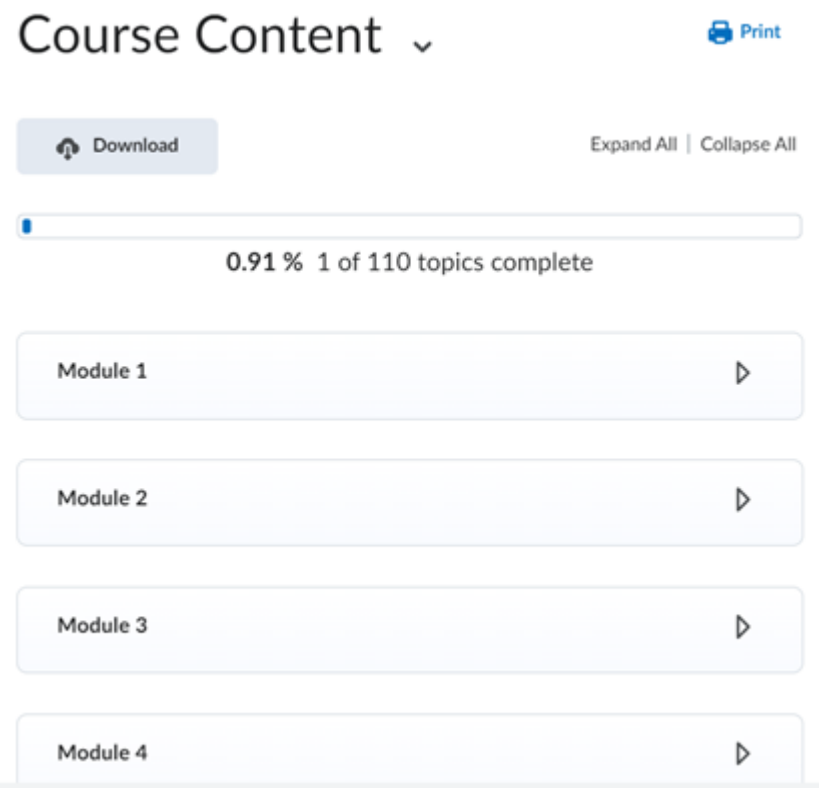
Finally, for a review of content creation watch the following video: **https://www.youtube.com/watch?v=ayYfHGYRuGA (https://www.youtube.com/watch?v=ayYfHGYRuGA)**

© Algonquin College 2022

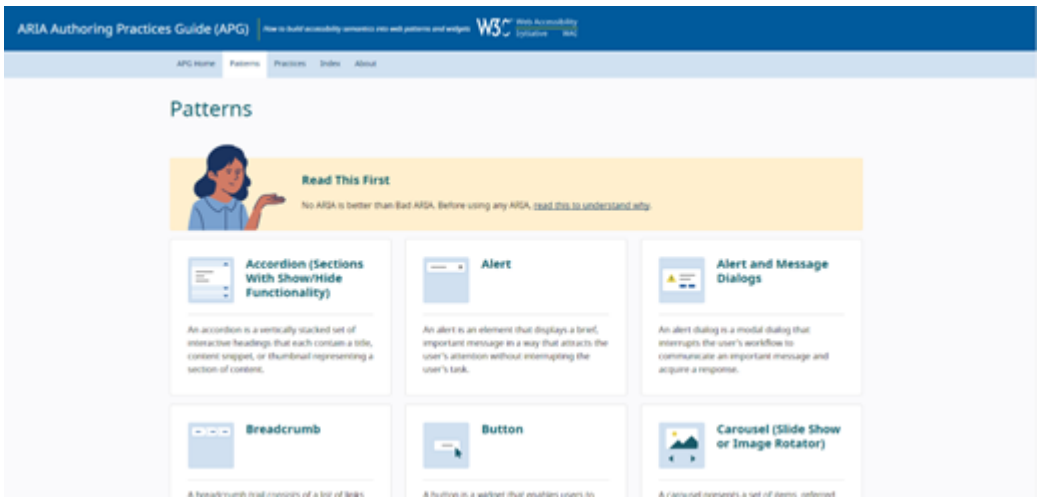
Accessibility patterns for guidance on semantic and keyboard behaviors

Code Patterns

When building accessible code you may run into certain visual styles you want to add that doesn't semantically have accessible features defined.
An example of this is an accordion style visual. Where the parent element able to collapse the child elements from view. See the image below



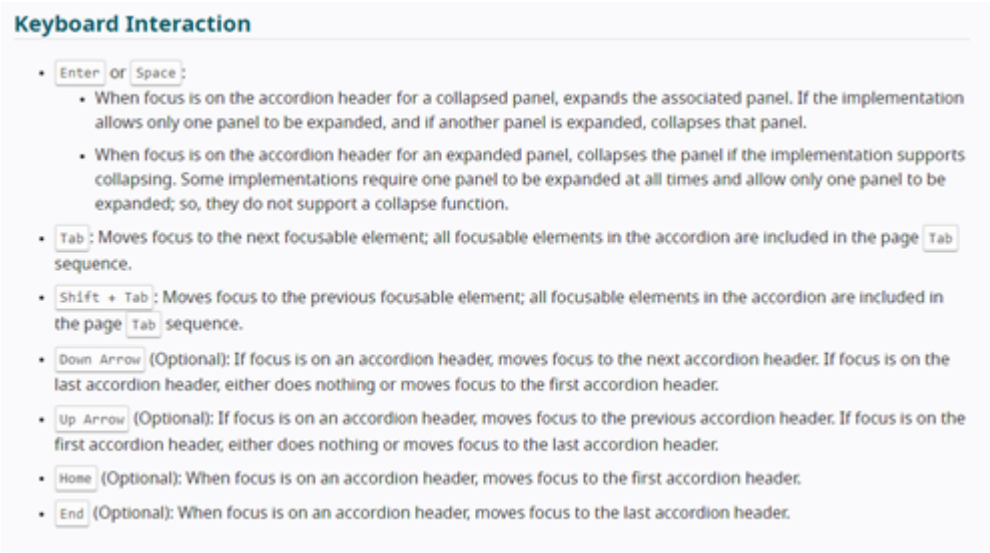
There is no standard HTML code for this as it's mostly achieved through JavaScript and CSS, the above image example uses div and span tags to create them.



https://www.w3.org/WAI/ARIA/apg/patterns/ (https://www.w3.org/WAI/ARIA/apg/patterns/)

The following link provides an in depth look at how to implement accessible patterns to the following code styles. It also shows what keyboard controls and semantics must be present in order for users to access it with keyboard controls.
Using the sample accordion design pattern review the following.

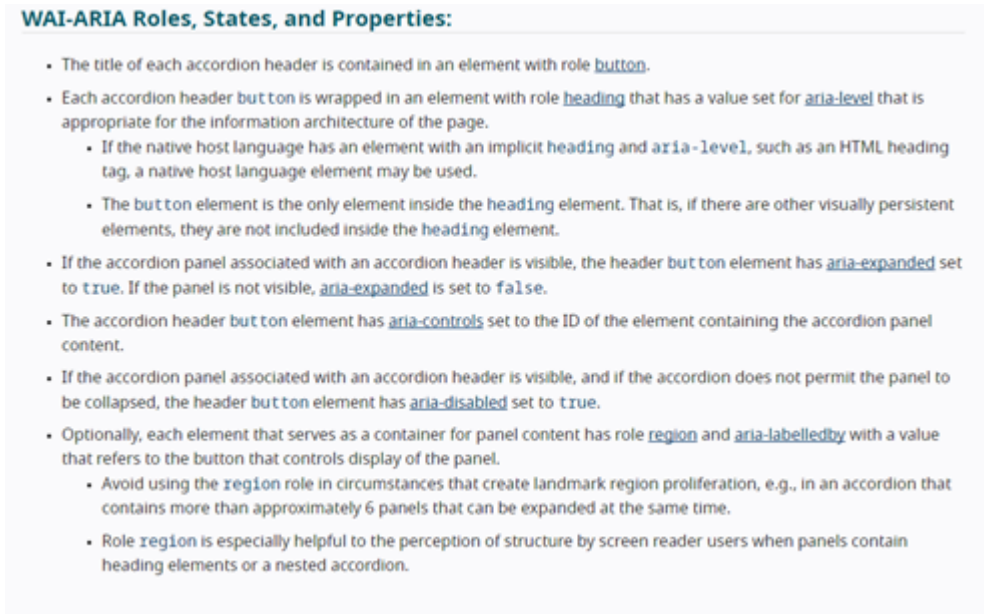
This images shows the acceptable keyboard interactions for that web pattern.



Keyboard Interaction

- `Enter` or `Space`:
  - When focus is on the accordion header for a collapsed panel, expands the associated panel. If the implementation allows only one panel to be expanded, and if another panel is expanded, collapses that panel.
  - When focus is on the accordion header for an expanded panel, collapses the panel if the implementation supports collapsing. Some implementations require one panel to be expanded at all times and allow only one panel to be expanded; so, they do not support a collapse function.
- `Tab`: Moves focus to the next focusable element; all focusable elements in the accordion are included in the page `Tab` sequence.
- `Shift + Tab`: Moves focus to the previous focusable element; all focusable elements in the accordion are included in the page `Tab` sequence.
- `Down Arrow` (Optional): If focus is on an accordion header, moves focus to the next accordion header. If focus is on the last accordion header, either does nothing or moves focus to the first accordion header.
- `Up Arrow` (Optional): If focus is on an accordion header, moves focus to the previous accordion header. If focus is on the first accordion header, either does nothing or moves focus to the last accordion header.
- `Home` (Optional): When focus is on an accordion header, moves focus to the first accordion header.
- `End` (Optional): When focus is on an accordion header, moves focus to the last accordion header.

The image below shows the acceptable Aria Roles, States, and properties



WAI-ARIA Roles, States, and Properties:

- The title of each accordion header is contained in an element with role button.
- Each accordion header `button` is wrapped in an element with role heading that has a value set for aria-level that is appropriate for the information architecture of the page.
  - If the native host language has an element with an implicit `heading` and `aria-level`, such as an HTML heading tag, a native host language element may be used.
  - The `button` element is the only element inside the `heading` element. That is, if there are other visually persistent elements, they are not included inside the `heading` element.
- If the accordion panel associated with an accordion header is visible, the header `button` element has aria-expanded set to `true`. If the panel is not visible, aria-expanded is set to `false`.
- The accordion header `button` element has aria-controls set to the ID of the element containing the accordion panel content.
- If the accordion panel associated with an accordion header is visible, and if the accordion does not permit the panel to be collapsed, the header `button` element has aria-disabled set to `true`.
- Optionally, each element that serves as a container for panel content has role region and aria-labelledby with a value that refers to the button that controls display of the panel.
  - Avoid using the `region` role in circumstances that create landmark region proliferation, e.g., in an accordion that contains more than approximately 6 panels that can be expanded at the same time.
  - Role `region` is especially helpful to the perception of structure by screen reader users when panels contain heading elements or a nested accordion.

Finally, for each pattern they show an example code snippet for you to review the code and see a live version to test.

Here is the accordion example discussed earlier.

Source: **https://www.w3.org/WAI/ARIA/apg/patterns/ (https://www.w3.org/WAI/ARIA/apg/patterns/)**

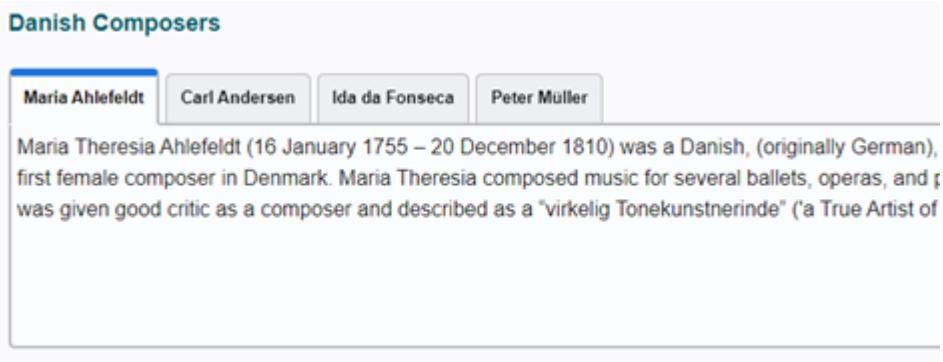
© Algonquin College 2022

Custom Widget

You have learned a little of what a widget is and in the previous section you looked at the design patterns of widgets for their semantic and keyboard behaviours.

When designing a custom widget there are several layers of improvements to make that widget accessible.

The example in this section you will be learning about tabs widget that is like a file folder with the topic at the top in tabs, then the body will hold information on that topic.



https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic/ (https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic/)

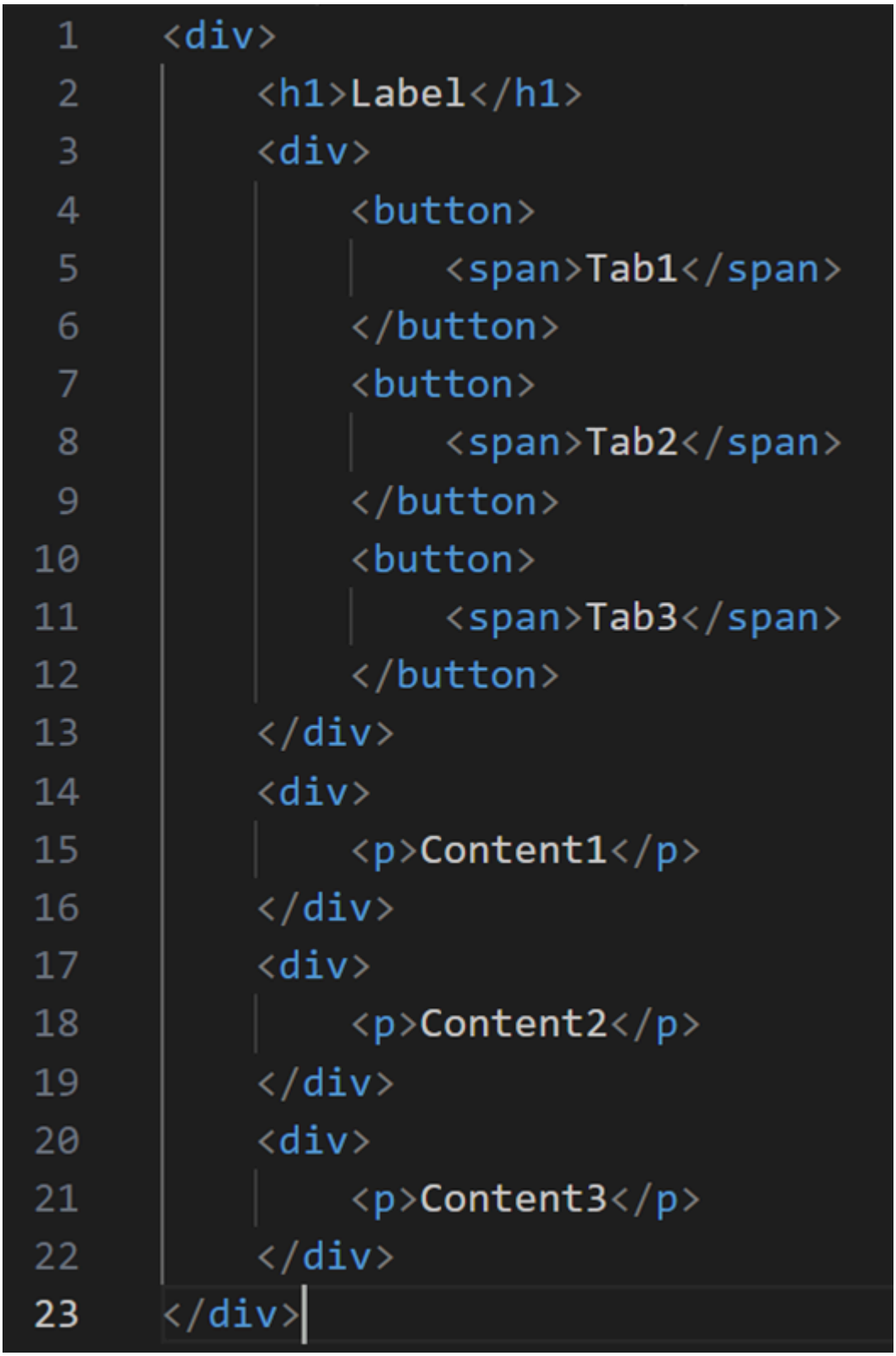
The Structure Layer

Since this widget is primarily build using divs and spans, these tags have no semantic meaning, which means they cannot be tabbed to, nor do they have any keyboard interactions.

This is a prime example of an HTML code pattern that needs Aria and accessible improvements.

To start, you need to build the base.

The base looks like the Image below



```
<div>
    <h1>Label</h1>
    <div>
        <button>
            <span>Tab1</span>
        </button>
        <button>
            <span>Tab2</span>
        </button>
        <button>
            <span>Tab3</span>
        </button>
    </div>
    <div>
        <p>Content1</p>
    </div>
    <div>
        <p>Content2</p>
    </div>
    <div>
        <p>Content3</p>
    </div>
</div>
```

The Functionality Layer

The functionality comes from the JavaScript which toggles what is currently being viewed based on the button that is clicked.

The code essentially creates a tablist class and applies it to each tab, then it adds click functionality and other event triggers. If one of the tablist items is selected, then it hides the previous one's tab panel while revealing its related tab panel.

A full breakdown of the code can be seen here (click the Open in Code Pen button) (https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic/#ex_label)

The Accessibility Layer

Once you have added functionality and have built it visually, you need to apply accessibility features.

A full list of accessibility features can be seen here (https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic/#accessibilityfeatures)

Essentially you will be adding the following:

- Roles
- Tab index
- Aria selected
- Aria controlled
- Focus indicator
- Keyboard accessibility
- Aria-hidden

To see a live implementation of it, watch video below You can finish it at the 21:17 mark where the demonstration ends.

- **https://youtu.be/F5IyeKFlJx4?t=607 (https://youtu.be/F5IyeKFlJx4?t=607)**

For further information, go to:
(https://www.w3.org/WAI/curricula/developer-modules/custom-widgets/)

- **https://www.w3.org/WAI/curricula/developer-modules/custom-widgets/ (https://www.w3.org/WAI/curricula/developer-modules/custom-widgets/) (https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic)**
- **https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic (https://www.w3.org/WAI/ARIA/apg/patterns/tabs/examples/tabs-automatic)**

© Algonquin College 2022

What is ARIA?

There are times when developing content for the web where finer controls are needed for accessible users. The Web Accessibility Initiative's (WAI) has developed a suite of frameworks to add attributes to HTML code for improved accessibility use. Accessible Rich Internet Application suite (ARIA) is that framework. ARIA helps describe navigation techniques to help those elements stand out when they may not syntactically have accessible controls built into their HTML elements.

ARIA is like a special set of instructions for HTML elements for accessible user's assistive technology (AT).

Compatibility

While Aria is more common today than ever, there are still cases where ARIA still is lacking. On older browsers, ARIA may not be supported. It is also not a requirement for all AT screen readers to be ARIA compatible. While ARIA helps give more access to the web for accessible users, it is always best to use semantic HTML over ARIA.

Common ARIA attributes

The following are common ARIA attributes and a description of what they do.

Role

aria-roledescription Describes an HTML element's role to a screen reader. This is great if an HTML tag doesn't fully describe what that element does.

Do not drink this

Labels

Sometimes an HTML element does not describe itself well enough. If needed you can apply an aria-label or aria-labelledby to an element to describe what it does.
For example if you are using an image as a button, a screen reader user cannot see that image, so they would get a description of "Button image" read out to them. They don't know what the button does.

<button aria-label="Close Window" ....

Aria-labelledby (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-labelledby) can delegate the label to another elements field by linking it' value to the element's ID. Examples can be see in the linked website.

ARIA States

Aria can announce states of elements in a webpage. Theses elements can be set in HTML or programmatically set using JavaScript.

Aria states include:

Aria-checked (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-checked) – announces if an element in a form has been checked

Aria-busy (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-busy) – States to aria-live that it should wait until an action is completed by the user before prompting

Aria-disabled (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-disabled) - Indicates that element and descendants are in a disabled state and should not be editable.

Aria-expanded (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-expanded) – Indicates if a section has been expanded or not.

Live Regions

Live regions were discussed in a previous module, but Aria-live regions gives updates to the user based on its politeness level.

It has 3 levels:

Aria-live="polite" – updates the user when the user has an inactive moment.

Aria-live="off" – Doesn't provideoe updates, unless user is focused on that element

Aria-live="assertive" – prompts the user right away

Widget Attributes

A widget is a small application that has a function or service. An example of a widget could be your phone's weather widget. It tells you the current weather, it's not a full application since it doesn't show you the weather to come for the day just a small snippet of info.

Another example on websites are the Chat boxes that allow you to communicate with a chat AI or staff member.

Since these are also embed into a website, it is important that you also apply accessibility to them as well.

This concept will be explored more in section 3.

Summary

Aria is a great option for when HTML itself falls short of provideoing certain accessibility options.

Always try to use semantic HTML when possible.

For a summary and demonstration of aria, watch the following video:
https://www.youtube.com/watch?v=g9Qff0b-lHk (https://www.youtube.com/watch?v=g9Qff0b-lHk)

For more information on aria uses, visit the following sites.

- **Beginner's guide to ARIA (https://www.lullabot.com/articles/what-heck-aria-beginners-guide-aria-accessibility)**
- **List of Aria attributes (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes)**

© Algonquin College 2022

What is Open-Source Software?

When it comes to software you have 2 categories of software licenses. open source and proprietary/commercial.

Your typical software that most people think of is software like Windows, Microsoft office, MacOS and Google Chrome. These tend to be proprietary licensed, which means the companies that make them own them. Their code is closed off and users are not allowed to copy the code or alter it.

Open source means that the code belongs to the authors/contributors, but also anyone who wants to use it (note: there are different levels of open-source licenses). The code is fully accessible by everyone, and it can be altered, used, or even sold by anyone.

There are lots of common open-source software used everyday such as:

- Mozilla Firefox
- Gimp art studios
- Linux
- VLC Media Player
- Python
- GNU Compiler
- PHP

Since anyone can work on open-source software and since open-source software is for everyone, there is a huge push for accessibility.

Read this article for more information on how open-source developers and communities are building more accessible software. (https://about.gitlab.com/blog/2021/04/07/how-the-open-source-community-can-build-more-accessible-products/)

Accessible Open-Source Tools and Software

As discussed above, the open-source community has been developing software to improve accessible user experience. Proprietary software tends to be really expensive to purchase a license for use.

An example is JAWs screen reader which at the time of writing costs $95 USD/year vs NVDA screen reader which is open source, free and always being updated, either by the developers who've originally made the software or by contributors who want to improve it.

There have been several open-source tools developed for accessible user or developer who want to build accessible software.

An extensive list of open-source accessible software can be found here. (https://www.accessibility.com/blog/free-and-open-source-applications-for-accessibility)

Accessible UI Component's Library

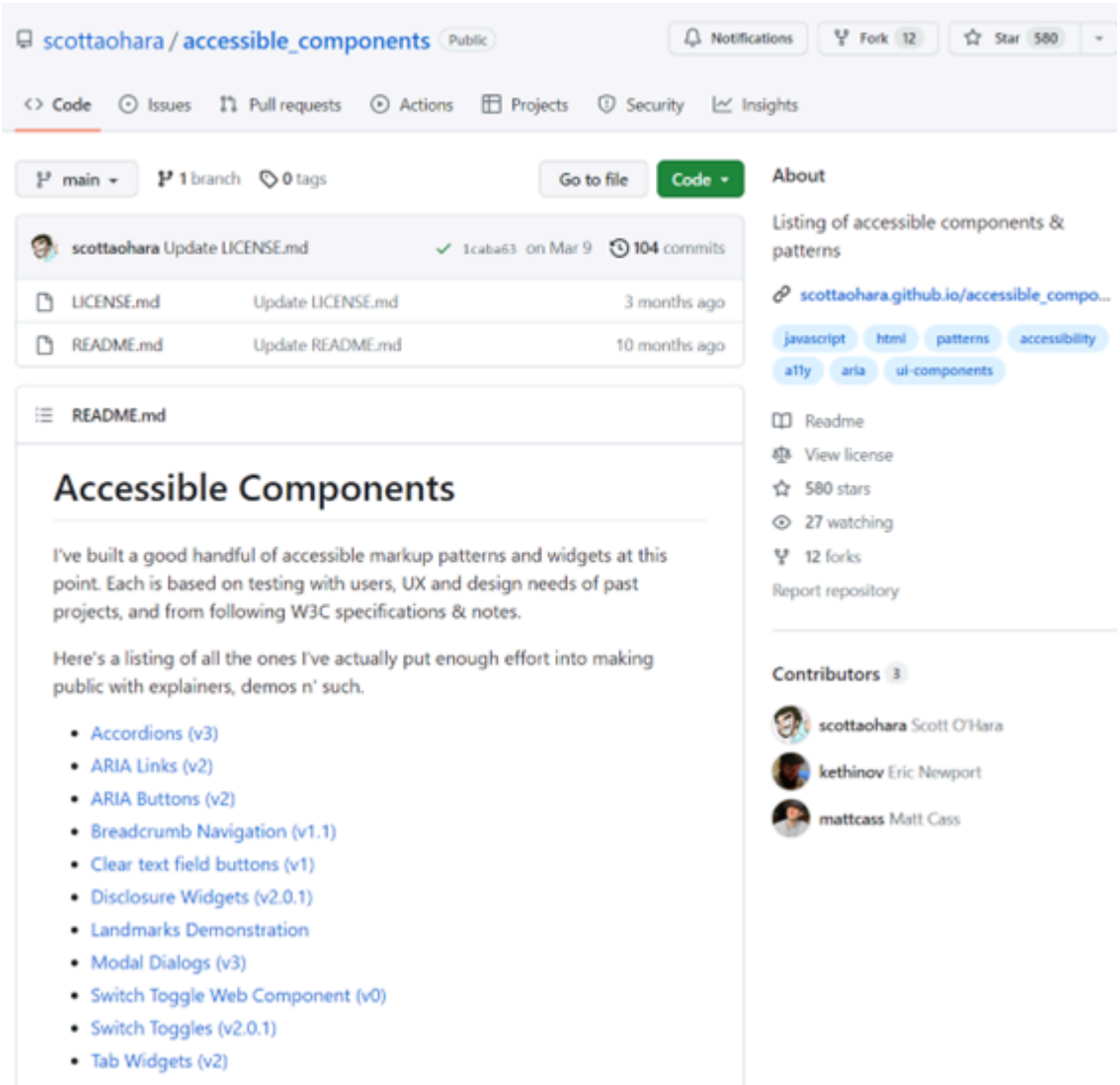
There are many open-source components and libraries you can use for/in development. Finding them can be a difficult task.

This link contains a list of UI components to use in your projects (https://www.digitala11y.com/accessible-ui-component-libraries-roundup/)

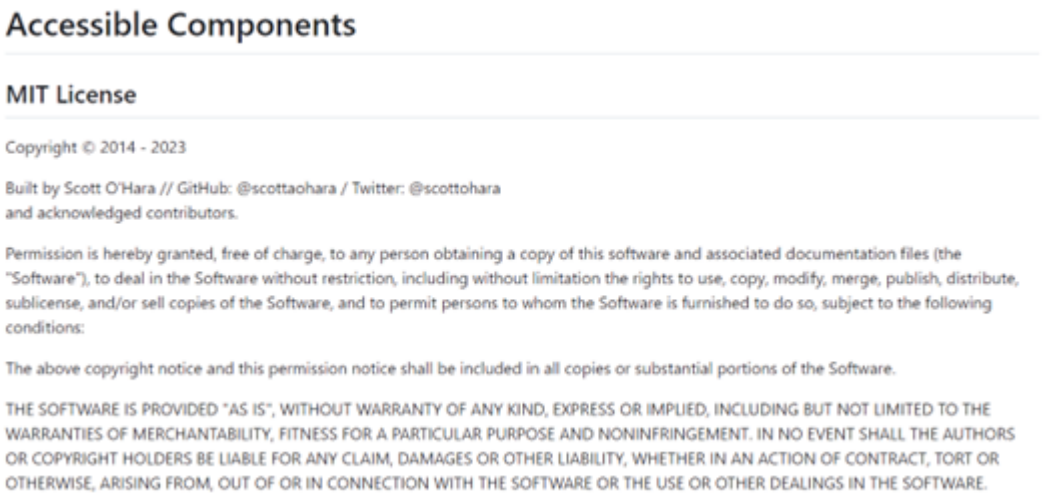
The list contains several examples and code snippets that you can use. Most of them will be free to use but always check the license for terms of use.

For example: Visit the following link that is titled: **Accessible Components by Scott O'Hara. (https://github.com/scottaohara/accessible_components)**

You should be taken to their GitHub repository's home page.



There will be a readme.md and License.md files listed. From the time of writing, the readme was updated 3 months ago which means it is sill fairly active.
Under the License.md you can see what the legal license is for using any copies of their code.
Which is a MIT license which states roughly that the software is free to do whatever you want with it so long as you include the same license with any portion of the code used.



Accessible Components

MIT License

Copyright © 2014 - 2023

Built by Scott O'Hara // GitHub: @scottaohara / Twitter: @scottohara
and acknowledged contributors.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

In summary, open-source components and libraries can help speed up your development time as it takes less time to build the same thing when someone has already done so and is giving it to you freely to use.

Commercial Software

There are times when you need to use commercial source software. This can be for compatibility or because there is no good open-source alternative.
For example if you want to build a website that integrates with Google voice controls, then you need to purchase a license from Google to use their API.
Since this is proprietary software, you are trying to link to then you have to purchase it for compatibility.

Note that Commercial Software and libraries are good as well. They have paid teams for support if anything is not working correctly or if you need help with integration, where in some cases open-source libraries may be very outdated and finding help can be difficult if there is not many contributors for that project.

For more information on open-source tools and components you can find a list on the w3's website. (https://www.w3.org/Status.html)

© Algonquin College 2022

Common content update issues caused by AJAX and JavaScript

Using AJAX

Using AJAX in a webpage to dynamically retrieve data is a great way for applications that need real time updates, or just for ease of flow in a website. However, discussed in the previous section there are some problems for screen reader users.

When applying AJAX to a website, ensure that you provide ways of telling the assistive technology that changes to content have been made.

Possible Problems

The way that a dynamic webpage loads, is typically a call to get a template HTML page, when that is loaded into the browser it then sends a request to the server to retrieve more data.

What could happen depending on the screen reader is that the template loads without the data populated.

If you are encountering a problem like this, a solution is to problem can be manually setting focus to an element of your choice when the page loads in.

While they are not using AJAX calls, the following Video has content and elements being loaded in after a user clicks the parent tab.

- **https://www.youtube.com/watch?v=srLRSQg6Jgg (https://www.youtube.com/watch?v=srLRSQg6Jgg)**

© Algonquin College 2022

As websites get more modern, the need for updatable content on a dynamic webpage grows. The use of AJAX(Asynchronous JavaScript and XML) for fetching data is used to make content dynamic. While dynamic content is great for visual users, more care needs to be coded in to allow for users with accessibility to access those pages. In the following module you will learn how to notify users that content has changed, what guidelines there are for timed content, how to set up a single page application, and what downsides there are for AJAX and dynamic webpages.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Notify accessible web users of content changes

02 Demonstrate compliance with WCAG's web-based time limits guidelines

03 Create dynamic web content while keeping web pages accessible

Key Terms and Concepts

1. AJAX: Asynchronous JavaScript and XML is code that allows a website to call a server and fetch data from it. A JSON string of data gets returned to the browser which then has to be programmatically added to the web page.

2. Hash: A form of encryption on data. It runs a text through a pattern to scramble their values.

3. SPA: A single page application that dynamically loads and changes content into it.

4. MPA: A multi-page application. This is a traditional website that serves multiple html pages to a user.

Notifying users of content change

Notifications

When building web pages there are times when a user must be notified of a change or progress on some of the elements in that webpage. For example, if you go to sign into a website you have to provide your username and password. What happens when you click the submit button, but you've added an incorrect password.

When you click the submit button, a call goes to a server sending the password and username. The username then needs to be matched to one in the database. After a match is found then the system needs to pull the password that belongs to that user.
It takes the stored password, which is **typically hashed (https://en.wikipedia.org/wiki/Cryptographic_hash_function)**, and then converts the sent password to a hashed password and compares their values. If a match is found, then the system can send a successful login message to create a successful session.
If the passwords don't match, an error message is then sent to the user's browser saying "incorrect username or password"

Logging in can take time since it has to go through several steps of validation. While it is doing this, the user will have to wait for the success or failure of the login. These logins are sometimes sent through an asynchronous server call without switching webpages. Once logged in the client's side of the website will update with the successful login or error message.

If a user is using a screen reader, how do they know if they successfully logged in or not?

A notification will have to be sent to a screen reader user when there has been a change or progress on what they are doing.
The same is true if a user is filling in a form and they incorrectly filled a field.

Notifying a Screen Reader

To notify a screen reader that there has been a change to the content there are 2 approaches:

1. Using role: status or alert
2. Using aria-live regions

This is another general-purpose page layout. Unlike the Basic page, this one includes samples of some commonly used page elements.

You can copy one, or more, of the specific elements from this page and paste them into other pages. By combining elements, you can create new layouts for your courses.

Status Role (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Roles/status_role)

When you apply a role of “status” it creates a “live region” that provides information to the user. This information is usually not drastically important, as if it was very serious the role would be labeled “alert”.

The default live value is “polite” and has an atomic value of “true”

Alert Role (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Roles/alert_role)

When you apply a role of “alert” that creates a “live region” that provides very important information to the user. This should only be used when the notification is used to get the user’s immediate attention.

- They are typically used if:
  an invalid value is entered on a form
- a timed event is about to expire
- a connection to a server was lost and changes cannot be saved
  The default live value of this role is “assertive”, and the atomic value is true.

Aria-live regions (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Live_Regions)

As discussed in a previous module, setting an element to have a ARIA live region. The live region allows for that section to announce changes to a screen reader based off of it’s aria-live value.

Values

Polite: announces to the screen reader when the users is inactive

Assertive: announces to the screen reader immediately

Off: turns the live region off and stops announcing.

For a visual example of applying roles and aria live regions watch the following video:
https://www.youtube.com/watch?v=RgVkOmgI0Zs (https://www.youtube.com/watch?v=RgVkOmgI0Zs)

Visit this webpage for an example of the “status” role in action and inspect the page using the developer tools on your browser to see how it is implemented.

Notifications comply with WCAG standards 4.1.3

Further Reading: **https://accessibility.huit.harvard.edu/provide-notification-dynamic-changes-content (https://accessibility.huit.harvard.edu/provide-notification-dynamic-changes-content)**

© Algonquin College 2022

Single page applications

What is a single page application?

Single Page Applications (SPA) are websites that only use a single html page. They dynamically change and alter their content using JavaScript and AJAX (Asynchronous JavaScript and XML) calls to the server. They act like native apps on devices rather than a traditional multi-page website.

Some examples of popular SPAs are Gmail, Google Maps, Airbnb, Netflix and many more.
For a quick explanation of how the webpage is made and the benefit of a SPA watch the following video: **https://youtu.be/eIxDHgzGCnY?t=77 (https://youtu.be/eIxDHgzGCnY?t=77)**

SPAs have a lot of benefits for server speed and load. **For a full breakdown of pros and cons, view this blog on SPA vs Multi Page Applications. (https://docs.astro.build/en/concepts/mpa-vs-spa/)**

Accessibility and SPAs

To summarize SPAs, they are a single web page that dynamically sends requests for more data to the server then uses the returned data and JavaScript to alter the appearance of that webpage.

For a thought experiment imagine a webpage about “Pets Pros and Cons”. This is a SPA and when navigating to the site, you have a single page that has several image cards with a prompt to “Choose your pet type”. See Image



When you click on one of the images, it requests any data or images of those animals from the server and builds a new webpage.

For visual users, the changes are apparent, and the webpage feels like a new webpage. However, to a screen reader user, no changes have happened. The page has the same title, and if they tab through the elements then they will see that the site is different, but it doesn't read out the webpage like a new page would.

The following issues arise when using SPAs for accessible users:

- What page are they on? (Theme and title)
- Has the page structure changed?
- Has any content changed on the page?
- Where is their keyboard focus?
- Are there any new HTML semantics that have changed?
- What happens if the user wants to go back a page (is history enabled)?

Going back to the example used. If the user clicks "Dogs" then the webpage should dynamically change the title from "Pet Info" to "Pet Info – Dogs", to notify the user that they are now in the dog section.

The page now looks like the image below



To improve the accessibility and notify the user that the page has changed, you can redirect focus to the heading “Dogs Information” by adding the HTML attribute tabindex=”-1” and the JavaScript focus method. Depending on how you are building the SPA, having an ARIA live-region setup can also help screen reader users note that the page has been changed. On the topic of ARIA, if there is any section where HTML5 semantics cannot be used, ensure you add aria labels and roles to inform the user what these areas of your code will be.

Finally, one feature of multipage applications that is very useful is the back button. For example if you were originally on Google.com, then navigated to your SPA Pet Information site, if you press the browser’s back button you navigate to google. If you are on google, then navigate to your Pet Information site, then click on “Dogs”, then press the browser’s back button, you would think you could be sent to the home page of pet information, but in actuality you get sent to google.

This is because when you clicked on “Dogs” you didn’t actually navigate your web history forward since there was no web page you went to next. The content dynamically changed, but the webpage is technically that initial webpage.

In order to solve this and have the website utilize history, you need to program your webpage using the “History API”. The history API simulates the forward and back buttons.

For further reading, visit the following webpages.
(https://www.tpgi.com/single-page-applications/)

- **https://www.tpgi.com/single-page-applications/ (https://www.tpgi.com/single-page-applications/)**
  (https://a11y-guidelines.orange.com/en/articles/single-page-app/)

- **https://a11y-guidelines.orange.com/en/articles/single-page-app/ (https://a11y-guidelines.orange.com/en/articles/single-page-app/)**
  (https://www.deque.com/blog/accessibility-tips-in-single-page-applications/)

- **https://www.deque.com/blog/accessibility-tips-in-single-page-applications/**
  (https://www.deque.com/blog/accessibility-tips-in-single-page-applications/)
  (https://johnsweetaccessibility.com/2020/05/accessibility-in-spas-part-1/)

- **https://johnsweetaccessibility.com/2020/05/accessibility-in-spas-part-1/**
  (https://johnsweetaccessibility.com/2020/05/accessibility-in-spas-part-1/)

For an in-depth talk on SPA accessible UI Navigation watch the following Video:
https://www.youtube.com/watch?v=ca2pKFwTsEI (https://www.youtube.com/watch?v=ca2pKFwTsEI)

© Algonquin College 2022

Time Limited Web Pages

Certain web pages are time sensitive, which means they need to reset after a set state of time. Usually a site's time is based off whether a user is active or inactive on that site.
Most of the time, a time limit is set to log off a user, so their account is not logged in if another user uses that device. Some examples of these sites are banks or government websites.
There can also be time limits on shopping/booking websites since they will reserve an item for a user, like a seat at a concert or a limited-edition item that is being ordered. Whatever the case is, users should have enough time to complete their business on that webpage.

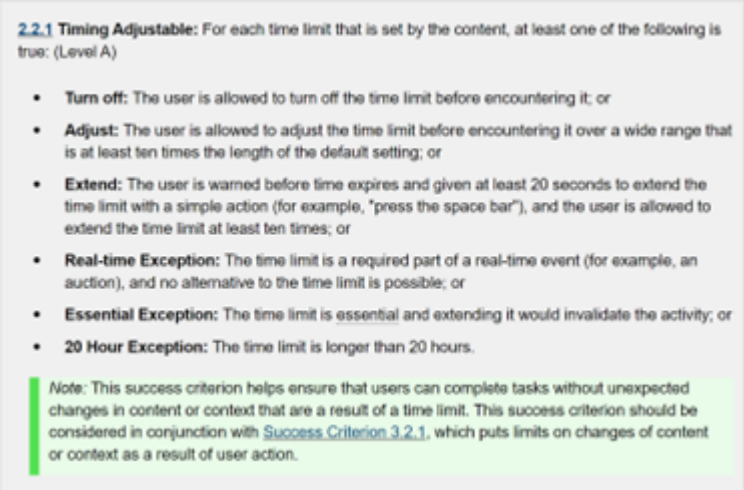
For most users the time limit doesn't matter too much, but some users who are using screen readers or ones that have low motor controls may need extra time.

Time limit Adjusting

To comply with the WCAG standards 2.2.1, a time limit should be able adjustable unless it falls under 3 sets of exceptions.

WCAG states your timer should be one of the following: Able to turn off, Adjustable, or Extendable unless the time-limit must be set in real-time (like an auction), essential like reflexive tests or timed actions, or is longer that 20hrs.

See a full breakdown in the image below



2.2.1 **Timing Adjustable:** For each time limit that is set by the content, at least one of the following is true: (Level A)

- **Turn off:** The user is allowed to turn off the time limit before encountering it; or
- **Adjust:** The user is allowed to adjust the time limit before encountering it over a wide range that is at least ten times the length of the default setting; or
- **Extend:** The user is warned before time expires and given at least 20 seconds to extend the time limit with a simple action (for example, "press the space bar"), and the user is allowed to extend the time limit at least ten times; or
- **Real-time Exception:** The time limit is a required part of a real-time event (for example, an auction), and no alternative to the time limit is possible; or
- **Essential Exception:** The time limit is essential and extending it would invalidate the activity; or
- **20 Hour Exception:** The time limit is longer than 20 hours.

Note: This success criterion helps ensure that users can complete tasks without unexpected changes in content or context that are a result of a time limit. This success criterion should be considered in conjunction with Success Criterion 3.2.1, which puts limits on changes of content or context as a result of user action.

https://www.w3.org/TR/UNDERSTANDING-WCAG20/time-limits-required-behaviors.html (https://www.w3.org/TR/UNDERSTANDING-WCAG20/time-limits-required-behaviors.html)

For an example on implementation of an extendable timer visit this WCAG tutorial (https://www.w3.org/TR/WCAG20-TECHS/SCR16.html)

© Algonquin College 2022

Accessibility features in mobile devices

Accessibility Features

Smartphones are very powerful these days and come with so many tools and services built in standard.

Let's look at some accessibility features of smartphones:

Voice control/Voice activation

"Okay Google" or "Hey Siri" have been commonplace since the 20-teens. Voice controls have been so key to allowing for hands free work between you and your phone. This can include:

- Calling by voice activation "Call James"
- Voice to text. Allows for notes and emails to be written.
- Recording voice notes
- Searching the internet "Hey Google, when is the next holiday?"
- Opening applications or navigation

Screen Reader

Both Android and iPhone have a native screen reader built into their operating systems. The screen readers can help with:

- Reading emails or text messages
- Browsing the internet
- Let you know if your screen is on or off.
- Describe what things are on your phone and how to interact with them.

Just take care if you have personal or private emails that you are not going through them in public.

Alter Text and Background

Adjusting how your phone looks if you are a user of low vison can help make a big difference.

- You can increase font size

- Change how font is presented
- Alter the text and background colour
- Enable dark mode or blue tint/eye comfort modes.

Zoom and Magnification

If text is too small, you can use phone's zoom mode to magnify the screen. This is typically pinch zoom but can be altered usually in the device's settings.

Subtitles

Most video media can have subtitles added to it to allow you to read the video's text. YouTube automatically generates captions if necessary.

Mono Audio

If a user only has hearing in one ear and has headsets, you can usually adjust how that audio is presented. Rather than having the audio divided among the two headsets you can allow it to be played through 1 ear.

Flashing Light Alerts

You can sometimes adjust how your phone presents alerts. If a user has difficulties hearing, a ring or vibrate may not help (if the phone is not in the pocket) You can set it to flash a Led light for notifications.

Delay for Touch and Hold Features

Smartphones allow you to alter how the tap or touch and holds are. Each system is different but are typically adjusted through settings.

Single Hand Controls

You can adjust your phone to make single hand controls easier, such as:

- Change screen size with single hand mode (android)
- Switch between left and right handers
- Single hand keyboard for iPhone
- Alter reachability on iPhones to allow for more access around the phone screen.

Change Button Controls

You can adjust your phone to use gestures to navigate and select if clicking buttons is not enough. If gestures are not enough, then swipes and taps can be used instead.

App Pinning

You can set a phone to stay locked in an app you want to use. This allows for other apps to not interfere, depending on your settings.

More Things

Your smart device has more features such as being able to connect with other smart devices or Bluetooth devices.

Things like:

- Keyboards
- Hearing aids
- Headsets/microphones
- Braille devices
- Smart speakers
- Etc.

How do you use your Smartphone? What apps draw directly from a cloud source?

© Algonquin College 2022

Coding considerations and testing tools for native mobile apps

Testing Mobile Apps

After building an app, you need to ensure that the application doesn't have any issues. When a build is complete on an app, smoke tests, regression tests and other tests must be performed on your application to see if any issues arise.

Much like websites, many tests can be automated to reduce the testing time needed.
Finding a good testing tool to test them is considered a big asset. Testing tools can range in price depending on your size and budget of the app itself.

Some free testing tools are as follows:

Appium (https://appium.io/docs/en/2.0/): A tool to test iOS, Android, and browsers. For more information watch: **https://www.youtube.com/watch?v=ormSbq-wAZc (https://www.youtube.com/watch?v=ormSbq-wAZc)**

Espresso (https://developer.android.com/training/testing/espresso#additional-resources-samples): A testing suite for android UI tests

Xcode UI (XCUI): (https://saucelabs.com/resources/blog/getting-started-with-xcuitest) A testing suite for iOS tests

Accessibility Tests

While developing apps, the platform you are developing for will have some built in accessibility checks. For example, if a button you are building in an android app is too small, the application will not compile. This is because its target size is too small to interact with. You will also get errors when there are incorrect contrast ratios on your elements.

Just like testing for web development, manual tests are very important. Once your app has passed the automated tests, run your app, and try turning on some accessibility controls to see how it runs. You should always double check content, alternative text values, and overall flow.

© Algonquin College 2022

Mobile screen readers i.e., Talkback & VoiceOver

Talkback

As seen in previous models, having a screen reader helps users who have low vision.
Talkback is a screen reader for Android; however, it does a little more than just reading the screen.

Talkback allows for swiping and tap controls to allow for easier navigation and interactions.

For a full breakdown of Talkback watch: **https://www.youtube.com/watch?v=_1yRVwhEv5I (https://www.youtube.com/watch?v=_1yRVwhEv5I)**

VoiceOver

Just like Android's Talkback, iPhones and iPads have Voice over.
It allows for screen reading and swipe tap controls.

For a full breakdown on how to set it up and use it watch this video:
https://www.youtube.com/watch?v=qDm7GiKra28 (https://www.youtube.com/watch?v=qDm7GiKra28)

© Algonquin College 2022

In this module you will learn how developing for mobile devices differs from web development. You will find out what accessibility tools are available to users who need them and how to test an application to ensure it will work for accessible users.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Identify the 2 major components of the OSX accessibility model

02 List accessibility features on mobile devices

03 Enable Talkback and Voiceover on an Android and iOS device respectively

04 Identify accessibility testing points on mobile devices to comply with WCAG guidelines

Key Terms and Concepts

- Accessibility Hierarchy: Mac OSX's structure on how an app is converted into an accessible version for the built in Assistive Technology.
- Regression Tests: Tests that run after updates to ensure fixes or changes haven't broken any other part of the app.
- Smoke Tests: Tests that test the software to see if it is ready for the next testing phase.
- Talkback: Android's built in screen reader software.
- VoiceOver: Apple's built in screen reader software.

OS X accessibility model & Android framework

Before we start this section, read the following document:
(https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html)

- https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html
(https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html)

OS X Accessibility Model

Apple's OS X has an accessibility model (similar to window's accessibility API) that allows a Native app to communicate with any of Apple's accessibility features.

Beginning in OS X 10.2, the accessibility architecture has helped user use their computers.
An example of this is how the VoiceOver app (a screen reader) helps read contents and provides cues on mouse and keyboard use.

OSX accessibility has 2 major components: communication interface & accessibility hierarchy

Communication:

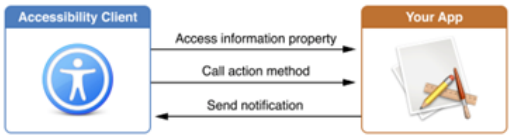
There are 3 ways the API interacts with an app you are developing.

Informational Properties: Provides information about your view or control. This is provided by the NSAccessibility protocol.

Action Methods: Provides methods of simulating button presses, mouse clicks and selections in your view or control. This is provided by the NSAccessibility protocol

Notifications: Provides a way for your application to notify the NSAccessibity protocol that a change has happened in your app.

The figure below shows communication between an app and the accessibility client



https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html
(https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html)

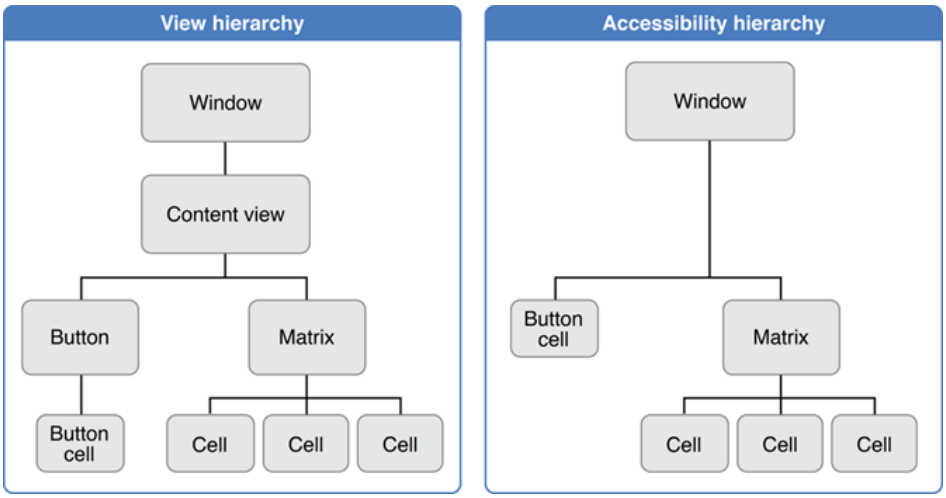
Accessibility Hierarchy

Accessibility objects in an application get arranged into a hierarchy for the accessibility model to process. He accessibility hierarchy removes some of the elements that are not needed for the accessible model.

This is similar to how a websites DOM gets converted into an Accessibility tree. Some of the parts of the DOM are removed as they aren't needed in the accessibility tree.

The figure below shows a hierarchy structure for a view and accessibility model.

As you can see in the diagram, the "button" and "content view" layers were removed as they were not needed for the accessibility model.



https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Accessibility/cocoaAXOverview/cocoaAXOverview.html (https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Accessibility/cocoaAXOverview/cocoaAXOverview.html)

For further, more in-depth reading on this subject, go to the following readings:

- **https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html (https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/OSXAXmodel.html)**
- **https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Accessibility/cocoaAXOverview/cocoaAXOverview.html (https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Accessibility/cocoaAXOverview/cocoaAXOverview.html)**

Android framework:

When developing for android, you can typically let the Android's native framework handle most accessibility.

There are times when you need to do adjustments in your code to improve accessibility for objects in your android app.
These typically come as adding more descriptions to button or form objects.

This video goes over some of the native accessibility service in android. It then goes over an example of how the accessibility framework communicates between an app and a user.

- **https://www.youtube.com/watch?v=LxKat_m7mHk (https://www.youtube.com/watch?v=LxKat_m7mHk)**

You should read more about this topic at:

- **https://developer.android.com/guide/topics/ui/accessibility (https://developer.android.com/guide/topics/ui/accessibility)**

© Algonquin College 2022

WCAG success criteria related to native mobile apps

The Web Accessibility Initiative (WAI) has released guidelines for mobile accessibility. This is typically an adjustment on the WCAG guideline to adjust for Mobile applications or Web applications on mobile devices.

The content is broken down into 4 Principles. (Counting offset to match the WCAG document)

2. Perceivable:

2.1 Small Screen Size
2.2 Zoom/Magnification
2.3 Contrast

3. Operable:

3.1 Keyboard Control for Touchscreen Devices
3.2 Touch Target Size and Spacing
3.3 Touchscreen Gestures
3.4 Device Manipulation Gestures
3.5 Placing buttons where they are easy to access

4. Understandable

4.1 Changing Screen Orientation (Portrait/Landscape)
4.2 Consistent Layout
4.3 Positioning important page elements before the page scroll
4.4 Grouping operable elements that perform the same action
4.5 Provide clear indication that elements are actionable
4.6 Provide instructions for custom touchscreen and device manipulation gestures

5. Robust

5.1 Set the virtual keyboard to the type of data entry required
5.2 Provide easy methods for data entry
5.3 Support the characteristic properties of the platform

To get a full description of each of the guidelines visit the WAI site. (https://www.w3.org/TR/mobile-accessibility-mapping/)

For techniques on how to implement the accessibility guidelines visit the WCAGA 2.0 Techniques (http://www.w3.org/WAI/GL/mobile-a11y-tf/MobileTechniques/)

© Algonquin College 2022

Future of Assistive Technology

To enable you to think about how you can be in the forefront of assistive technology, consider the future and begin to have a vision of the goals you are trying to incorporate in your cloud computing career.

What is the Goal of Future AT?

The goal of assistive technology is and will always be to allow all users an easy / easier and affordable method to interact with their environment.

Technology is growing quick, and people are always looking to develop new things to improve the quality of life for everyone. Since 3d printers started, prosthetics have been easier to create and develop. They also have been cheaper to make for users, enabling greater accessibility. On the medical front, there is excited discussions around 3D printing an actual transplantable kidney soon. And speaking of which, doctors are now experimenting with remote surgery using 5G connections and robotic hands and eyes that they can control.

At the time of this course's creation (2023) the current trend is AI large language models. How can AI and large language models help users access the internet?

- AI helps improve:
- voice recognition/voice controls
- virtual assistants
- code markup for developing
- automated captions/translations on video or audio
- image recognition for describing an image to low vision users
- eye controls
- document revision
- automated sign language generator

Some cool projects in the works:

- Sign Language to text translations, slaitAI, signallAI (Videoeo example of slait in action)
- Text to sign language generator

Further Reading: For a more comprehensive and exciting vision of potential future technology, read the following articles:

- **https://www.boia.org/blog/3-ways-that-artificial-intelligence-can-improve-web-accessibility (https://www.boia.org/blog/3-ways-that-artificial-intelligence-can-improve-web-accessibility)**

- **https://news.microsoft.com/en-ca/2023/05/18/ai-is-strengthening-the-future-of-accessibility/ (https://news.microsoft.com/en-ca/2023/05/18/ai-is-strengthening-the-future-of-accessibility/) (https://www.atera.com/blog/how-accessibility-tech-could-change-the-future/)**

- **https://www.atera.com/blog/how-accessibility-tech-could-change-the-future/ (https://www.atera.com/blog/how-accessibility-tech-could-change-the-future/) (https://medium.com/@roger.mc/artificial-intelligence-and-digital-accessibility-a-match-made-in-heaven-dc1b5afbb03c)**

- **https://medium.com/@roger.mc/artificial-intelligence-and-digital-accessibility-a-match-made-in-heaven-dc1b5afbb03c (https://medium.com/@roger.mc/artificial-intelligence-and-digital-accessibility-a-match-made-in-heaven-dc1b5afbb03c) (https://www.linkedin.com/pulse/how-generative-ai-being-used-enhance-accessibility-inclusion-alston)**

- **https://www.linkedin.com/pulse/how-generative-ai-being-used-enhance-accessibility-inclusion-alston (https://www.linkedin.com/pulse/how-generative-ai-being-used-enhance-accessibility-inclusion-alston)**

© Algonquin College 2022

Important people to follow for accessibility information

As you complete this course, remember that accessibility issues do not simply stop. The requirements and technology to assist greater accessibility are constantly being updated. This is a never-ceasing learning opportunity for you. Some people started out in computer programming or Cloud computing as a technician or software engineer and eventually recognized a niche in accessibility that nobody else was addressing. They have gone on to develop accessibility services or awareness and have become household names in the industry. Here are a few of these:

Léonie Watson

Léonie is Director of TetraLogical (an accessibility services Company), part of the W3C Board of Directors, and co-Chair of the W3C Web Applications Working Group. She is a leader in web accessibility and co-organiser of the inclusive design 24 conference.

A great talk about how to improve performance of accessible sites can be found here: **https://www.youtube.com/watch?v=tQkPog-stj8 (https://www.youtube.com/watch?v=tQkPog-stj8)**

For more about Léonie you can visit her website.

Rob Dodson

Rob was an accessibility developer for google and hosted the A11ycasts YouTube channel where he teaches common accessibility tips and semantics.
You can find out more about Rob on his website.

Jamie Teh & Mick Curran

Jamie and Mick are a pair of Australian developers who created the NVDA screen reader. Both of them are blind, so they have been personally affected by slight barriers to everyday tasks.

- **A great news article can be found here. (https://www.abc.net.au/news/2023-06-05/mick-curran-jamie-teh-changing-lives-of-the-blind-with-nvda/101768788)**
- **More information about NVDA can be found here. (https://www.nvaccess.org/about-nv-access/)**

© Algonquin College 2022

In your final module, you will be learning about some of the people in the industry pushing accessible technology forward, what the future of accessible technology looks like and how WCAG is changing their guidelines with their newest version of WCAG 3.0

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Discuss what the future of accessible technology may look like

02 Assess how open-source technology is improving assistive technology

03 List changes in WCAG 3.0 from WCAG 2.2

Key Terms and Concepts

- LLM: Large Language Models. This is an AI technology based on neural networks for processing language. ChatGPT is an example of an LLM
- Open Source: Software, texts, or ideas that have been made available to anyone free of charge and can be redistributed and modified, depending on the licensing statements.
- **Web of Things: (https://www.w3.org/WoT/)** A set of standards to be applied to any internet-based technology

Open-source Projects

While MS Office, Adobe, and many other software charge a fee for use, there are many other projects that are considered open-source. This is to say that the original source code is made freely available and may be modified to suit specific purposes. Many of these open-source packages can be changed to suit specific needs.

Linux is a good example of this. Linux was created as an open source code package. Unfortunately, when it first began to be used, it was very difficult for the average person to even try to install it. They would be met with questions during the install, or see messages like “Fatal Error” and panic immediately. However, some groups began working with the installation packages and found a way to make the installation as seamless as MS Windows. Now, anyone can install Linux on their computers and it comes with a host of free applications and accessibility options. Software coders have been encouraged to create other applications and share them.

Accessibility and Open-Source

Open-source projects are great since they are free for users/clients to use them. They can be updated fast, and anyone can bring their experience to the project.

The following website has a great list on accessible technology that is open sourced. (https://www.accessibility.com/blog/free-and-open-source-applications-for-accessibility)

Anyone can contribute to a project. The open-source community is a great way to meet new developers, build your coding skills and contribute to a great cause.

You can even start contributing now since there are many ways you can help a project, from bug testing/reproduction, bug fixes, new feature development and even writing documentation or translation services.

There are even open-source projects from large companies like Microsoft and Google that you can work on.

© Algonquin College 2022

WCAG 3.0

While you learned about WCAG 2.2 and WCAG 2.3 is currently being revised and should release soon, WCAG 3.0 has already begun development.

What WCAG 3.0 improves, in short, is to expand guidelines and recommendations so that more users with disability barriers have access to improved accessibility on sites and also expands what technologies are covered under the WCAG umbrella.

WCAG 3.0 helps users with speech troubles work with digital voice-controlled systems, and also covers “Web of Things” technology as well as virtual and augmented reality.

For a quick breakdown on the development process of WCAG 3.0 watch: **https://www.youtube.com/watch?v=V7OUqkAH1wM (https://www.youtube.com/watch?v=V7OUqkAH1wM)**

- **For the full WCAG documentation guidelines view the WAI website here. (https://www.w3.org/TR/wcag-3.0/)**

© Algonquin College 2022